

DirectX 11 で学ぶ CG と流体シミュレーション

「山から粒子が落ちる」プログラムで読み解く
3D グラフィックス・地形生成・SPH・表面生成

対象プログラム: water_from_mountain (C++ / DirectX 11 / HLSL)

https://github.com/githubtarohario/water_from_mountain

2026 年 9 月

本書について

本書のねらい

本書は、1本の完結したプログラム「山から粒子が落ちる」を教材として、3D コンピュータグラフィックス（CG）と粒子法による流体シミュレーションを**基礎から**解説する教科書です。このプログラムは、岩肌の山に刻まれた谷を水の粒子が流れ落ち、粒子の集まりから水面を生成して描画します。見た目はシンプルですが、その中には CG と物理シミュレーションの主要な技術がひとつとおり詰まっています。

- 座標系・ベクトル・行列と、3D をスクリーンに映すまでの**変換の数学**
- GPU が絵を作る手順（**レンダリングパイプライン**）と DirectX 11 の使い方
- GPU 上で動く小さなプログラム、シェーダー（HLSL）
- 光と影、テクスチャによる**質感表現**
- ノイズによる**地形・テクスチャの手続き的生成**、外部モデルの取り込み
- 粒子で水を表す **SPH 法**と、それを速く安定に動かす工夫
- 粒子の集まりを 1 枚の水面として見せる**スクリーンスペース流体レンダリング**

各技術は「なぜ必要か → 考え方 → 数式 → 実際のコード」の順に説明します。掲載しているコードは本書の生成時にソースファイルから自動的に抜き出しており、リポジトリの実装と一致しています。

想定する読者

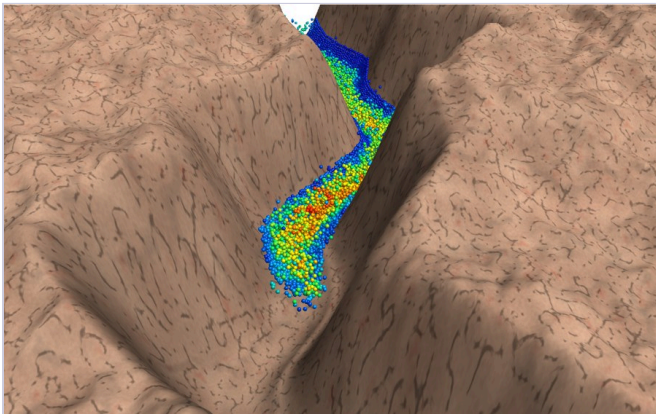
C++ の基本文法（クラス、参照、STL の `vector`）が読めること、高校数学（三角関数、ベクトル）を一度は学んだことを前提にします。行列や微分は必要になった箇所です。DirectX やシェーダーの経験は不要です。

本書の構成

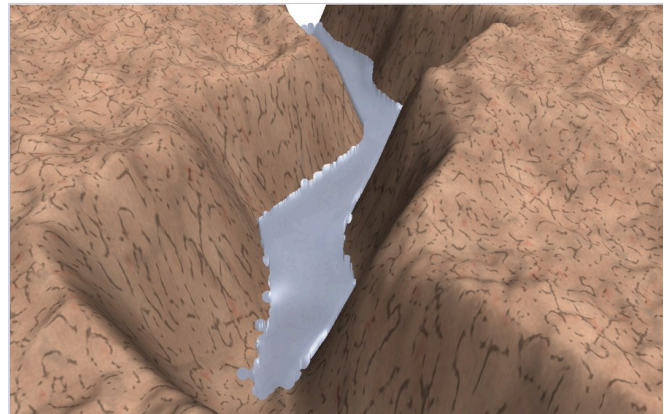
部	内容
第 I 部 基礎編	3D の数学、レンダリングパイプラインと DirectX 11、シェーダー、ライティングとテクスチャ
第 II 部 地形編	ノイズ、高さマップ地形、テクスチャの生成と読み込み、Blender からのメッシュ取り込み
第 III 部 流体編	粒子法の考え方、SPH の理論と実装、近傍探索、境界と安定化、並列化
第 IV 部 表現編	粒子の描画（スフィア・インポスター）、表面生成、カメラと操作、メインループ
第 V 部 発展編	性能とデバッグ、発展課題
付録	数学のまとめ、ファイル構成、用語集、参考文献

プログラムの入手と実行

リポジトリ	https://github.com/githubtarohario/water_from_mountain
必要環境	Windows 10/11、DirectX 11 対応 GPU、Visual Studio 2019/2022（C++ デスクトップ 開発）
ビルド	<code>MountainParticles.sln</code> を Release x64 でビルド、または <code>build.bat</code>
操作	左ドラッグ: 回転 / 右ドラッグ: 移動 / ホイール: ズーム / 1: 粒子表示 / 2: 表面生成 / Space: 一時停止 / R: リセット



粒子表示モード（速度で青→緑→赤に着色）



表面生成モード（粒子から水面を作って描画）

本書の記法

- ベクトルは太字 ($\mathbf{v}$)、スカラーは斜体 (x)、行列は大文字斜体 (M) で表します。
- コードの引用は緑の枠で、その上にファイル名を示します。
- `ポイント` は必ず押さえてほしい要点、`注意` は間違えやすい点、`コラム` は補足の読み物です。
- 各章末の**演習**は、プログラムを改造して理解を確かめる課題です。

目次

本書について

第 I 部 基礎編

- 第 1 章 3D グラフィックスの数学 — 座標系・ベクトル・行列・変換
- 第 2 章 レンダリングパイプラインと DirectX 11
- 第 3 章 シェーダー入門 — HLSL で GPU をプログラムする
- 第 4 章 ライティングとテクスチャ

第 II 部 地形編

- 第 5 章 ノイズと手続き的生成
- 第 6 章 高さマップ地形の設計と生成
- 第 7 章 テクスチャの生成と画像ファイルの読み込み
- 第 8 章 外部メッシュの取り込み — OBJ と Blender

第 III 部 流体編

- 第 9 章 粒子で流体を表す — 粒子法の考え方
- 第 10 章 SPH の理論と実装
- 第 11 章 近傍探索 — 空間ハッシュとカウンティングソート
- 第 12 章 境界処理・安定化・エミッタ
- 第 13 章 OpenMP による並列化

第 IV 部 表現編

- 第 14 章 粒子の描画 — インスタンスリングとスフィア・インポスター
- 第 15 章 表面生成 — スクリーンスペース流体レンダリング
- 第 16 章 カメラ・入力・メインループ

第 V 部 発展編

- 第 17 章 性能とデバッグ
- 第 18 章 発展課題

付録

- 付録 A 数学のまとめ
- 付録 B ファイル構成と API 要約
- 付録 C 用語集
- 付録 D 参考文献

第 1 部

基礎編

3D の数学、GPU が絵を作る仕組み、シェーダー、光と質感。
すべての土台となる知識を、このプログラムのコードを見ながら学びます。

第 1 章

3D グラフィックスの数学 — 座標系・ベクトル・行列・変換

3D 空間の点をどうやって画面の 2D に映すのか。この章では、CG のあらゆる場面で使う「座標系」「ベクトル」「行列」と、ワールド → ビュー → 射影という変換の連鎖を、このプログラムのカメラのコードを題材に学びます。

1.1 座標系 — 右手系と左手系

3D 空間の位置は (x, y, z) の 3 つの数で表します。ただし「どちらが上か」「奥行きはどちらが正か」には流儀があり、これを座標系と呼びます。本プログラムは DirectX の慣習に従い、**Y 軸が上**、**左手系**を使います。

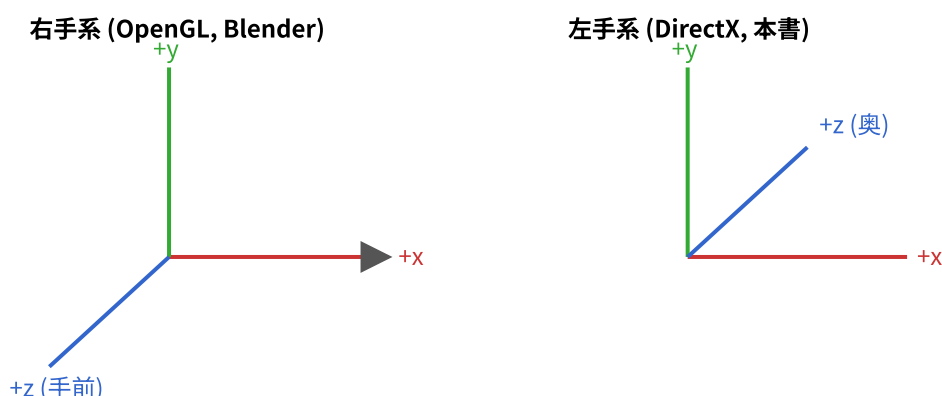


図 1-1 右手系と左手系。x を右、y を上にしたとき、z が手前を向くか奥を向くかが違う

左手系では、左手の親指を $+x$ 、人差し指を $+y$ に向けたとき、中指が $+z$ を指します。カメラは $+z$ の方向を見る（奥ほど z が大きい）ため、**奥行き** = z と素直に対応するのが利点です。Blender は右手系・ Z が上なので、モデルを持ち込むときは変換が必要です（第 8 章）。

ポイント 座標系の違いは「符号の向き」だけですが、外積の向き、三角形の表裏（巻き順）、回転の正方向のすべてに影響します。本書では左手系で統一し、右手系のデータを扱うときは明示的に変換します。

1.2 ベクトルの演算

ベクトルは「大きさと向き」を持つ量で、位置・速度・力・法線などすべてベクトルで表します。CG で頻出する演算は次の 4 つです。

演算	定義	CG での用途
加減・スカラー倍	$(a_x \pm b_x, \dots)$ 、 ka	位置の移動、速度の積分 $\mathbf{x} \leftarrow \mathbf{x} + \mathbf{v}\Delta t$
長さ・正規化	$ \mathbf{a} = \sqrt{a_x^2 + a_y^2 + a_z^2}$ 、 $\hat{\mathbf{a}} = \mathbf{a}/ \mathbf{a} $	距離判定、方向だけを取り出す（法線・光源方向）
内積	$\mathbf{a} \cdot \mathbf{b} = a_x b_x + a_y b_y + a_z b_z = \mathbf{a} \mathbf{b} \cos \theta$	2 ベクトルのなす角、明るさ（ $\mathbf{N} \cdot \mathbf{L}$ ）、射影
外積	$\mathbf{a} \times \mathbf{b} = (a_y b_z - a_z b_y, a_z b_x - a_x b_z, a_x b_y - a_y b_x)$	2 ベクトルに垂直なベクトル（面の法線）、三角形の表裏判定

内積は特に重要です。単位ベクトル同士の内積は $\cos \theta$ そのもので、「どれくらい同じ向きか」を $-1 \sim 1$ で表します。第 4 章のライティングでは、面の法線 $\mathbf{N}$ と光の方向 $\mathbf{L}$ の内積で明るさを決めます。

外積は「2 つのベクトルが張る平面に垂直なベクトル」を返し、その長さは平行四辺形の面積に等しくなります。地形メッシュの法線計算（第 6 章、第 8 章）や三角形の向きの判定に使います。左手系では、 $\mathbf{a}$ から $\mathbf{b}$ へ左手の指を曲げたとき親指の向きが $\mathbf{a} \times \mathbf{b}$ です。

プログラムでは DirectXMath ライブラリの `XMVECTOR`（SIMD レジスタに載る 4 成分ベクトル）と `XMFLOAT3`（メモリ格納用）を使い分けます。例えばカメラのパン操作では、視線方向と上方向の外積で「右方向」を作っています。

src/Camera.cpp

```
//-----
// Pan
// アルゴリズム:
// カメラの前方ベクトル forward = normalize(target - pos) を求め、
// right = normalize(cross(worldUp, forward)), up = cross(forward, right)
// で画面の右・上方向を作り、その方向に注視点をずらす。
//-----
void Camera::Pan(float dx, float dy)
{
    XMFLOAT3 posF = GetPosition();
    XMVECTOR pos = XMLoadFloat3(&posF);
    XMVECTOR target = XMLoadFloat3(&m_target);
    XMVECTOR forward = XMVector3Normalize(XMVectorSubtract(target, pos));
    XMVECTOR worldUp = XMVectorSet(0.0f, 1.0f, 0.0f, 0.0f);
    XMVECTOR right = XMVector3Normalize(XMVector3Cross(worldUp, forward));
    XMVECTOR up = XMVector3Cross(forward, right);

    target = XMVectorAdd(target, XMVectorScale(right, dx));
    target = XMVectorAdd(target, XMVectorScale(up, dy));
    XMStoreFloat3(&m_target, target);
}
```

1.3 行列と変換

「点を移動する」「回転する」「拡大縮小する」といった操作は、 4×4 の行列 M を点に掛けることで表せます。3D なのに 4×4 を使うのは、平行移動を掛け算で表すため、点は $(x, y, z, 1)$ の 4 成分（同次座標）として扱います。

平行移動 $T(t) : (x, y, z, 1) \rightarrow (x+t_x, y+t_y, z+t_z, 1)$

拡大縮小 $S(s) : (x, y, z, 1) \rightarrow (s_x x, s_y y, s_z z, 1)$

y 軸回転 $R_y(\theta) : (x, z) \rightarrow (x \cos \theta + z \sin \theta, -x \sin \theta + z \cos \theta)$

行列の便利な性質は**合成**できることです。「回転してから移動」は R と T の積 1 つで表せ、何千個の頂点にも同じ 1 つの行列を掛ければ済みます。ただし積の順序を入れ替えると結果が変わります（回転してから移動 $\neq$ 移動してから回転）。

行ベクトルと列ベクトル DirectXMath は「行ベクトル $\times$ 行列」($\mathbf{v}' = \mathbf{v}M$) の規約で、変換は左から右へ $\mathbf{v} W V P$ と並びます。数学の教科書や OpenGL は「行列 $\times$ 列ベクトル」($\mathbf{v}' = M\mathbf{v}$) で、順序が逆になります。どちらも同じ計算ですが、行列をメモリに置く順（行優先 / 列優先）と組み合わせると混乱のもとになります。第 3 章で GPU に渡すときの「転置」として再登場します。

1.4 変換の連鎖 — ワールド・ビュー・射影

3D の頂点が画面に映るまでには、3 つの座標系を経由します。

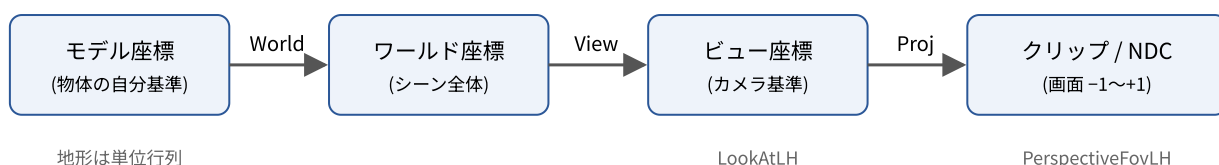


図 1-2 頂点が画面に映るまでの座標変換

- **ワールド行列 W :** 物体をシーン内に置く（位置・向き・大きさ）。本プログラムの地形は最初からワールド座標で頂点を作るため単位行列（変換なし）です。
- **ビュー行列 V :** 「カメラが原点にあって +z を向いている」状態になるよう、世界全体を動かす行列。
`XMMatrixLookAtLH(位置, 注視点, 上方向)` で作ります。
- **射影行列 P :** 遠近感を付け、画面の範囲を $-1 \sim +1$ に正規化する。`XMMatrixPerspectiveFovLH(画角, アスペクト比, near, far)`。

src/Camera.cpp

```
//-----  
// GetViewMatrix  
//-----  
XMMATRIX Camera::GetViewMatrix() const  
{  
    XMFLOAT3 posF = GetPosition();  
    XMVECTOR pos = XMLoadFloat3(&posF);  
    XMVECTOR target = XMLoadFloat3(&m_target);  
    XMVECTOR up = XMVectorSet(0.0f, 1.0f, 0.0f, 0.0f);  
    return XMMatrixLookAtLH(pos, target, up);  
}
```

src/Camera.cpp

```
//-----  
// GetProjMatrix  
//-----  
XMMATRIX Camera::GetProjMatrix(float aspect) const  
{  
    return XMMatrixPerspectiveFovLH(m_fovY, aspect, m_nearZ, m_farZ);  
}
```

透視投影の仕組み

遠くのものほど小さく見える「遠近感」は、ビュー座標の x, y を奥行き z で割ることで生まれます。射影行列はこの割り算の準備をする行列で、変換後の 4 成分目 w に z が入ります。GPU はラスタライズの直前に自動的に $x/w, y/w, z/w$ を計算します（透視除算）。

$$x_{\text{ndc}} = x_{\text{view}} \cdot P_{00} / z_{\text{view}}, \quad y_{\text{ndc}} = y_{\text{view}} \cdot P_{11} / z_{\text{view}} \\ P_{00} = 1 / (\text{aspect} \cdot \tan(\text{fov}_y/2)), \quad P_{11} = 1 / \tan(\text{fov}_y/2)$$

この式は第 15 章で「画面の深度から 3D 位置を逆算する」ときに、そのまま逆向きに使います。

深度値

変換後の z/w は 0（near 面）～1（far 面）の深度値になり、手前の物体が奥の物体を隠す判定（深度テスト、第 2 章）に使われます。深度値は z に対して線形ではなく、近くに精度が集中します。near を小さくしすぎると遠くの精度が落ちて「Z ファイティング」（面がちらつく）が起きるため、本プログラムでは near = 0.5、far = 400 としています。

1.5 オービットカメラ

本プログラムのカメラは、注視点の周りを回る「オービットカメラ」です。カメラの位置を極座標（水平角 θ 、仰角 φ 、距離 D ）で持ち、そこから直交座標を計算します。

$$\text{pos} = \text{target} + D(\cos\varphi \sin\theta, \sin\varphi, \cos\varphi \cos\theta)$$

src/Camera.cpp

```
//-----  
// GetPosition  
// 極座標 (yaw, pitch, distance) → 直交座標  
//-----  
XMVECTOR Camera::GetPosition() const  
{  
    const float cp = std::cos(m_pitch);  
    return XMVECTOR(   
        m_target.x + m_distance * cp * std::sin(m_yaw),  
        m_target.y + m_distance * std::sin(m_pitch),  
        m_target.z + m_distance * cp * std::cos(m_yaw));  
}
```

マウスのドラッグ量を θ, φ の増分に変換すれば回転、ホイールで D を倍率で変えればズームになります。 φ を $\pm 89^\circ$ に制限しているのは、真上（ 90° ）で視線と上方向ベクトルが平行になり LookAt が壊れるのを防ぐためです。

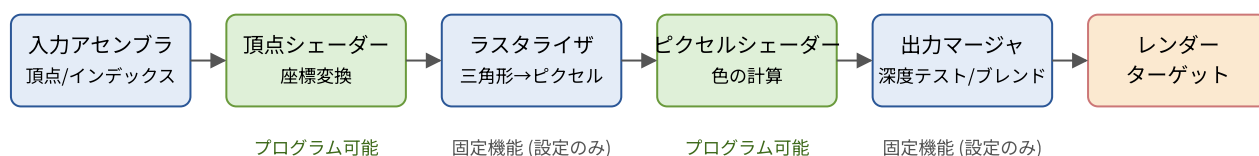
演習 1

1. `Camera.cpp` の初期 pitch を 10° に変えて、谷を低い視点から眺めてみましょう。透視投影による遠近感がどう変わりますか。
2. `GetProjMatrix` の画角 (`m_fovY`) を 90° にすると何が起きるか観察し、なぜそうなるか P_{00} , P_{11} の式から説明してください。
3. `near` を 0.01 にして遠くの地形をよく見ると、面のちらつきが起きるか確認してください。

レンダリングパイプラインと DirectX 11

GPU は「頂点の集まり」をどのような手順で「画面のピクセル」に変えるのでしょうか。この章ではその手順（レンダリングパイプライン）を学び、DirectX 11 という API でそれをどう操作するのかを、初期化コード `Graphics.cpp` に沿って理解します。

2.1 レンダリングパイプラインの全体像



頂点シェーダーは「頂点ごと」、ピクセルシェーダーは「ピクセルごと」に GPU の何千ものコアで並列に実行される

図 2-1 レンダリングパイプライン（本プログラムで使う段のみ）

1. **入力アセンブラ (IA)**: 頂点バッファとインデックスバッファから頂点を読み、三角形に組み立てる。
2. **頂点シェーダー (VS)**: 頂点ごとに実行される小さなプログラム。主に座標変換（第 1 章）を行い、クリップ座標を出力する。
3. **ラスタライザ (RS)**: 透視除算、ビューポート変換、裏面カリングを行い、三角形が覆うピクセルを列挙する。頂点の属性（法線、UV など）はピクセル位置に応じて補間される。
4. **ピクセルシェーダー (PS)**: ピクセルごとに実行され、色を計算する。ライティングやテクスチャはここで行う。
5. **出力マージャ (OM)**: 深度テストで隠面を除去し、ブレンドで既存の色と合成してレンダーターゲットに書く。

ポイント GPU が速い理由は「同じ小さなプログラムを、大量のデータに対して並列に走らせる」構造にあります。頂点シェーダーとピクセルシェーダーは、隣の頂点や隣のピクセルのことを知らずに独立して動きます。この制約を理解すると、シェーダーで「できること・できないこと」が見えてきます。

2.2 深度テスト、カリング、ブレンド

深度テスト

手前の物体が奥の物体を隠すには、ピクセルごとに「今まで描いた中で一番手前の深度」を覚えておく必要があります。これが**深度バッファ**です。新しいピクセルの深度が記録より小さい（手前）ときだけ色と深度を更新します。本プログラムでは 24bit 深度 + 8bit ステンシルの `D24_UNORM_S8_UINT` 形式を使い、比較関数は `LESS` です。

裏面カリング

閉じた物体の裏側の面は見えないので、描画前に捨てると速くなります。三角形の表裏は、画面上での頂点の並び順（巻き順）で判定します。DirectX の既定では「画面上で時計回り」が表です。地形メッシュは上から見て時計回りになるよう頂

点を並べており（第 6 章）、粒子のビルボードや全画面パスではカリングを無効にしています。

ブレンド

半透明の物体は、既に描かれている色と混ぜて書き込みます。もっとも基本的なアルファブレンドは次式です。

$$\text{result} = \text{src} \cdot \alpha + \text{dst} \cdot (1 - \alpha)$$

本プログラムでは、水面を地形の上に $\alpha = 0.9$ で合成するとき（第 15 章）に使います。

2.3 DirectX 11 の主要オブジェクト

DirectX 11（Direct3D 11）は、GPU を操作するための API です。登場するオブジェクトは多いですが、役割で分類すると整理できます。

オブジェクト	役割
ID3D11Device	GPU の「工場」。バッファ・テクスチャ・シェーダー・ステートなどのリソースを作る
ID3D11DeviceContext	GPU への「命令書」。リソースをパイプラインにセットして描画命令を出す
IDXGISwapChain	画面に表示するバッファの管理。描き終わった裏バッファを表に出す（Present）
バッファ / テクスチャ	GPU 上のメモリ。頂点、インデックス、定数、画像などのデータ本体
ビュー（RTV / DSV / SRV）	リソースを「どの用途で使うか」を示す窓口。同じテクスチャを描画先（RTV）としても読み取り元（SRV）としても使える
ステート（RS / DS / Blend / Sampler）	固定機能段の設定をまとめたオブジェクト。作っておいて切り替える
シェーダー	コンパイル済みの GPU プログラム

初期化の流れ

Graphics::Initialize は、デバイスとスワップチェーンを同時に作り、バックバッファ用のレンダーターゲットビュー、深度バッファ、共通ステートを準備します。

src/Graphics.cpp

```
//-----
// Initialize
//   アルゴリズム（初期化手順）:
//     1. スワップチェーンの設定（DXGI_SWAP_CHAIN_DESC）を用意
//     2. D3D11CreateDeviceAndSwapChain でデバイスとスワップチェーンを同時作成
//     3. バックバッファ RTV と深度バッファ DSV を作成
//     4. 共通ステートを作成
//-----
bool Graphics::Initialize(HWND hWnd, int width, int height)
{
    m_width  = width;
    m_height = height;

    // ---- 1. スワップチェーンの設定 ----
    DXGI_SWAP_CHAIN_DESC sd = {};
```

```

sd.BufferCount      = 2;                                // ダブルバッファ
sd.BufferDesc.Width  = width;
sd.BufferDesc.Height = height;
sd.BufferDesc.Format = DXGI_FORMAT_R8G8B8A8_UNORM;      // 8bit RGBA
sd.BufferDesc.RefreshRate.Numerator   = 60;
sd.BufferDesc.RefreshRate.Denominator = 1;
sd.BufferUsage       = DXGI_USAGE_RENDER_TARGET_OUTPUT;
sd.OutputWindow      = hWnd;
sd.SampleDesc.Count   = 1;                                // マルチサンプルなし
sd.SampleDesc.Quality = 0;
sd.Windowed           = TRUE;
sd.SwapEffect         = DXGI_SWAP_EFFECT_DISCARD;

// ---- 2. デバイスとスワップチェーンの作成 ----
UINT createFlags = 0;
#ifdef _DEBUG
createFlags |= D3D11_CREATE_DEVICE_DEBUG; // デバッグビルドでは D3D のデバッグレイヤーを有効化
#endif
D3D_FEATURE_LEVEL featureLevels[] = { D3D_FEATURE_LEVEL_11_0 };
D3D_FEATURE_LEVEL obtained = D3D_FEATURE_LEVEL_11_0;

HRESULT hr = D3D11CreateDeviceAndSwapChain(
    nullptr, // 既定のアダプタ
    D3D_DRIVER_TYPE_HARDWARE, // ハードウェア GPU を使う
    nullptr,
    createFlags,
    featureLevels, 1,
    D3D11_SDK_VERSION,
    &sd,
    m_swapChain.GetAddressOf(),
    m_device.GetAddressOf(),
    &obtained,
    m_context.GetAddressOf());

if (FAILED(hr))
{
    // デバッグレイヤーが入っていない環境ではフラグなしで再試行する
    createFlags &= ~D3D11_CREATE_DEVICE_DEBUG;
    hr = D3D11CreateDeviceAndSwapChain(nullptr, D3D_DRIVER_TYPE_HARDWARE, nullptr, createFlags,
        featureLevels, 1, D3D11_SDK_VERSION, &sd,
        m_swapChain.GetAddressOf(), m_device.GetAddressOf(), &obtained, m_context.GetAddressOf());
    if (FAILED(hr))
    {
        MessageBoxW(hWnd, L"D3D11 デバイスの作成に失敗しました。", L"エラー", MB_OK | MB_ICONERROR);
        return false;
    }
}

// ---- 3. RTV / DSV ----
if (!CreateBackBufferViews())
    return false;

// ---- 4. 共通ステート ----
if (!CreateCommonStates())
    return false;

return true;
}

```

注目点をいくつか挙げます。

- `D3D_FEATURE_LEVEL_11_0` を要求しています。これは Shader Model 5.0 と、後で使う構造化バッファ・インスタンスングを保証する水準です。
- Debug ビルドではデバッグレイヤーを有効にします。API の誤用（例えば SRV にバインドしたままのテクスチャへ描画する）を出力ウィンドウに警告してくれる、開発時の強力な味方です。SDK のデバッグコンポーネントが無い環境では失敗するので、その場合はフラグなしで作り直しています。
- `ComPtr` は COM オブジェクトのスマートポインタで、スコープを抜けると自動で `Release` します。DirectX のリソース管理はこれに任せるのが定石です。

ステートオブジェクト

固定機能段の設定は「ステートオブジェクト」として事前に作り、描画時に切り替えます。本プログラムでは次の組み合わせを用意しています。

src/Graphics.cpp

```
//-----
// CreateCommonStates
//-----
bool Graphics::CreateCommonStates()
{
    HRESULT hr;

    // ラスタライザ: 塗りつぶし、裏面（時計回り以外）をカリング
    D3D11_RASTERIZER_DESC rd = {};
    rd.FillMode          = D3D11_FILL_SOLID;
    rd.CullMode           = D3D11_CULL_BACK;
    rd.FrontCounterClockwise = FALSE;    // 時計回りを表面とする（左手系）
    rd.DepthClipEnable    = TRUE;
    hr = m_device->CreateRasterizerState(&rd, m_rsSolid.GetAddressOf());
    if (FAILED(hr)) return false;

    // ラスタライザ: カリングなし（向きを気にしなくてよいビルボードやフルスクリーン三角形用）
    rd.CullMode = D3D11_CULL_NONE;
    hr = m_device->CreateRasterizerState(&rd, m_rsNoCull.GetAddressOf());
    if (FAILED(hr)) return false;

    // 深度ステート: 通常（テストあり・書き込みあり）
    D3D11_DEPTH_STENCIL_DESC dd = {};
    dd.DepthEnable      = TRUE;
    dd.DepthWriteMask   = D3D11_DEPTH_WRITE_MASK_ALL;
    dd.DepthFunc        = D3D11_COMPARISON_LESS;
    hr = m_device->CreateDepthStencilState(&dd, m_dsDefault.GetAddressOf());
    if (FAILED(hr)) return false;

    // 深度ステート: 無効（フルスクリーンの後処理パス用）
    dd.DepthEnable      = FALSE;
    dd.DepthWriteMask   = D3D11_DEPTH_WRITE_MASK_ZERO;
    hr = m_device->CreateDepthStencilState(&dd, m_dsDisabled.GetAddressOf());
    if (FAILED(hr)) return false;

    // ブレンド: 不透明
    D3D11_BLEND_DESC bd = {};
    bd.RenderTarget[0].BlendEnable = FALSE;
    bd.RenderTarget[0].RenderTargetWriteMask = D3D11_COLOR_WRITE_ENABLE_ALL;
    hr = m_device->CreateBlendState(&bd, m_bsOpaque.GetAddressOf());
    if (FAILED(hr)) return false;
```

```

// ブレンド: アルファブレンド result = src * srcAlpha + dst * (1 - srcAlpha)
bd.RenderTarget[0].BlendEnable = TRUE;
bd.RenderTarget[0].SrcBlend = D3D11_BLEND_SRC_ALPHA;
bd.RenderTarget[0].DestBlend = D3D11_BLEND_INV_SRC_ALPHA;
bd.RenderTarget[0].BlendOp = D3D11_BLEND_OP_ADD;
bd.RenderTarget[0].SrcBlendAlpha = D3D11_BLEND_ONE;
bd.RenderTarget[0].DestBlendAlpha = D3D11_BLEND_ZERO;
bd.RenderTarget[0].BlendOpAlpha = D3D11_BLEND_OP_ADD;
hr = m_device->CreateBlendState(&bd, m_bsAlpha.GetAddressOf());
if (FAILED(hr)) return false;

// サンプラー: 線形補間 + ミップマップ + 繰り返し (地形テクスチャ用)
D3D11_SAMPLER_DESC sdesc = {};
sdesc.Filter = D3D11_FILTER_ANISOTROPIC;
sdesc.MaxAnisotropy = 8;
sdesc.AddressU = D3D11_TEXTURE_ADDRESS_WRAP;
sdesc.AddressV = D3D11_TEXTURE_ADDRESS_WRAP;
sdesc.AddressW = D3D11_TEXTURE_ADDRESS_WRAP;
sdesc.MaxLOD = D3D11_FLOAT32_MAX;
hr = m_device->CreateSamplerState(&sdesc, m_sampLinearWrap.GetAddressOf());
if (FAILED(hr)) return false;

// サンプラー: 補間なし + 端で固定 (深度テクスチャ用)
sdesc.Filter = D3D11_FILTER_MIN_MAG_MIP_POINT;
sdesc.MaxAnisotropy = 1;
sdesc.AddressU = D3D11_TEXTURE_ADDRESS_CLAMP;
sdesc.AddressV = D3D11_TEXTURE_ADDRESS_CLAMP;
sdesc.AddressW = D3D11_TEXTURE_ADDRESS_CLAMP;
hr = m_device->CreateSamplerState(&sdesc, m_sampPointClamp.GetAddressOf());
if (FAILED(hr)) return false;

return true;
}

```

2.4 フレームの流れ

1 フレームの描画は「クリア → 描画命令 → Present」の繰り返しです。

src/Graphics.cpp

```

//-----
// BeginFrame
//-----
void Graphics::BeginFrame(const float clearColor[4])
{
    ID3D11RenderTargetView* rtv = m_backBufferRTV.Get();
    m_context->OMSetRenderTargets(1, &rtv, m_depthDSV.Get());
    m_context->ClearRenderTargetView(rtv, clearColor);
    m_context->ClearDepthStencilView(m_depthDSV.Get(), D3D11_CLEAR_DEPTH | D3D11_CLEAR_STENCIL, 1.0f,
0);

    // ビューポートは Resize で変わる可能性があるので毎フレーム設定し直す
    D3D11_VIEWPORT vp = {};
    vp.Width = static_cast<float>(m_width);
    vp.Height = static_cast<float>(m_height);
    vp.MinDepth = 0.0f;
    vp.MaxDepth = 1.0f;
}

```

```
m_context->RSSetViewports(1, &vp);
}
```

Present の引数 1 は**垂直同期**（モニタのリフレッシュに合わせる）を意味し、画面の裂け（ティアリング）を防ぎます。代わりにフレームレートはモニタの周波数（60 Hz や 120 Hz）で頭打ちになります。

2.5 CPU から GPU へのデータ転送

GPU のメモリは CPU から直接触れません。用途に応じて 3 種類の作り方を使い分けます。

Usage	特徴	本プログラムでの用途
IMMUTABLE	作成時にデータを渡し、以後変更不可。最速	地形の頂点・インデックスバッファ
DEFAULT	GPU が読み書き。CPU からは <code>UpdateSubresource</code> でコピー	テクスチャ、深度テクスチャ
DYNAMIC	CPU が毎フレーム書き換える。 <code>Map</code> / <code>Unmap</code>	定数バッファ、粒子の <code>StructuredBuffer</code>

src/Graphics.cpp

```
//-----
// UpdateBuffer
//   D3D11_MAP_WRITE_DISCARD: GPU が前の内容を使い終わるのを待たずに
//   新しいメモリ領域を割り当ててもらふ（毎フレーム更新するバッファの定石）
//-----
void Graphics::UpdateBuffer(ID3D11Buffer* buf, const void* data, size_t byteSize)
{
    D3D11_MAPPED_SUBRESOURCE mapped = {};
    if (SUCCEEDED(m_context->Map(buf, 0, D3D11_MAP_WRITE_DISCARD, 0, &mapped)))
    {
        memcpy(mapped.pData, data, byteSize);
        m_context->Unmap(buf, 0);
    }
}
```

`D3D11_MAP_WRITE_DISCARD` は「前の内容は捨ててよいので、新しいメモリ領域をすぐ割り当ててほしい」という指定です。GPU がまだ前フレームのデータを読んでいる最中でも CPU が待たされないため、毎フレーム更新するバッファの定石です。

演習 2

1. クリア色（`main.cpp` の `clearColor`）を空色に変えてみましょう。
2. 地形の描画で `GetRasterizerNoCull()` を使うようにし、地形の裏側（下から見上げる）が描かれるようになることを確認してください。次に、カメラを地形の下に潜らせて違いを観察しましょう。
3. 深度ステートを `GetDepthStateDisabled()` にして地形を描くと、何が起きますか。描画順との関係を考えてください。

シェーダー入門 — HLSL で GPU をプログラムする

シェーダーは GPU 上で走る小さなプログラムです。この章では HLSL の基本文法、頂点シェーダーとピクセルシェーダーの書き方、CPU からデータを渡す定数バッファ、そして本プログラムが多用する「頂点バッファを使わない描画」を学びます。

3.1 HLSL の基本

HLSL (High Level Shading Language) は C に似た文法で、`float3`、`float4x4` のようなベクトル・行列型が組み込まれています。よく使う要素を挙げます。

要素	例と意味
ベクトル型	<code>float2 float3 float4 int2 uint</code> 。 <code>v.xyz</code> , <code>v.xy</code> のように成分を自由に取り出せる (スウィズル)
行列型と積	<code>float4x4 M</code> ; <code>mul(v, M)</code> 。 <code>M._m00</code> で要素にアクセス
組み込み関数	<code>dot cross normalize length saturate lerp pow exp sqrt clamp</code>
セマンティクス	<code>: POSITION</code> , <code>: SV_POSITION</code> など。入出力の「意味」をパイプラインに伝えるラベル
リソース	<code>Texture2D</code> , <code>SamplerState</code> , <code>StructuredBuffer<T></code> , <code>cbuffer</code>
レジスタ	<code>register(b0)</code> (定数), <code>register(t0)</code> (テクスチャ/バッファ), <code>register(s0)</code> (サンプラー)。CPU 側の <code>PSSetShaderResources(0, ...)</code> の番号と対応

3.2 頂点シェーダーとピクセルシェーダー

もっとも基本的な例として、地形のシェーダーを見ます。頂点シェーダーは頂点を受け取り、クリップ座標 (`SV_POSITION`) と、ピクセルシェーダーに渡したい属性を出力します。

```
shaders/Terrain.hlsl

//-----
// 頂点シェーダーの入力 (C++ 側の Terrain::Vertex と対応)
//-----
struct VSIn
{
    float3 pos : POSITION;    // ワールド座標
    float3 nrm : NORMAL;     // 法線
    float2 uv  : TEXCOORD0;  // テクスチャ座標
};
```

```
shaders/Terrain.hlsl

//-----
// 頂点シェーダー → ピクセルシェーダー
//-----
struct PSIn
```

```
{
    float4 posH : SV_POSITION;    // クリップ座標
    float3 posW : TEXCOORD0;      // ワールド座標
    float3 nrm : TEXCOORD1;       // 法線 (ワールド空間)
    float2 uv : TEXCOORD2;        // テクスチャ座標
};
```

shaders/Terrain.hlsl

```
//-----
// 関数名 : VSMain
// 概要   : 頂点をクリップ空間へ変換し、法線と UV をそのまま渡す
//          (地形のワールド行列は単位行列なので変換不要)
// 引数   : i : 頂点データ
// 戻り値 : PSIn
//-----
PSIn VSMain(VSIn i)
{
    PSIn o;
    o.posH = mul(float4(i.pos, 1.0), ViewProj);
    o.posW = i.pos;
    o.nrm = i.nrm;
    o.uv = i.uv;
    return o;
}
```

`SV_POSITION` は「システム値」セマンティクスで、ラスタライザが位置として使います。それ以外の `TEXCOORD n` は自由な用途の属性で、三角形内で線形補間されてピクセルシェーダーに届きます。ピクセルシェーダーは補間された属性から色を計算し、`SV_Target` (レンダーターゲットへの出力) を返します。詳しい陰影の計算は第 4 章で扱います。

3.3 定数バッファ — CPU から GPU へ値を渡す

ビュー行列や光源方向のように「フレーム中は一定でシェーダー全体から参照する値」は、**定数バッファ**にまとめて渡します。本プログラムは 1 つの構造体 `FrameConstants` をすべてのシェーダーで共有しています。

src/FrameConstants.h

```
struct FrameConstants
{
    DirectX::XMFLOAT4X4 View;      // ワールド → ビュー変換行列 (転置済み)
    DirectX::XMFLOAT4X4 Proj;      // ビュー → クリップ変換行列 (転置済み)
    DirectX::XMFLOAT4X4 ViewProj;  // View * Proj (転置済み)
    DirectX::XMFLOAT4 CameraPosW;  // xyz: カメラのワールド座標
    DirectX::XMFLOAT4 LightDirW;   // xyz: ワールド空間の「光源へ向かう」単位ベクトル
    DirectX::XMFLOAT4 LightDirV;   // xyz: 同じベクトルをビュー空間に変換したもの
    DirectX::XMFLOAT4 ScreenSize;  // x: 幅, y: 高さ, z: 1/幅, w: 1/高さ (ピクセル)
    DirectX::XMFLOAT4 Params;      // x: 粒子描画半径, y: 経過時間[秒], z,w: ぼかし方向 (1,0) or (0,1)
};
```

shaders/Common.hlsli

```
//-----
// CBFram : 1 フレームごとの定数 (register b0)
// 行列は CPU 側で転置済みなので mul(float4(v,1), Matrix) の形で使う。
//-----
cbuffer CBFram : register(b0)
{
```

```

float4x4 View;      // ワールド → ビュー
float4x4 Proj;      // ビュー → クリップ
float4x4 ViewProj;  // ワールド → クリップ
float4   CameraPosW; // xyz: カメラのワールド座標
float4   LightDirW;  // xyz: ワールド空間の光源方向 (光源へ向かう単位ベクトル)
float4   LightDirV;  // xyz: ビュー空間の光源方向
float4   ScreenSize; // x: 幅, y: 高さ, z: 1/幅, w: 1/高さ
float4   Params;     // x: 粒子半径, y: 時間, z,w: ぼかし方向
};

```

16 バイト境界 定数バッファは 16 バイト (float4) 単位で並べる規則があります。float3 の後に float を置くと詰められますが、float3 を 2 つ続けると 2 つ目は次の 16 バイトに送られ、C++ 側の構造体とずれます。本プログラムはすべて float4 / float4x4 で構成して、この問題を避けています。

行列の転置

第 1 章で触れたとおり、DirectXMath は行優先で $\mathbf{vM}$ の規約、HLSL の既定は列優先パッキングです。行列をそのまま転送すると HLSL は転置された行列として読んでしまいます。そこで CPU 側で転置してから転送すると、HLSL 側で自然に `mul(float4(v,1), M)` と書けます。

src/main.cpp

```

//-----
// 関数名 : UpdateFrameConstants
// 概要   : カメラと光源からフレーム定数を組み立てて GPU に転送する
// 引数   : なし (g_app を使用)
// 戻り値 : なし
//-----
void UpdateFrameConstants()
{
    Graphics& gfx = g_app.gfx;
    FrameConstants& fc = g_app.frame;

    const float aspect = static_cast<float>(gfx.GetWidth()) / static_cast<float>(gfx.GetHeight());
    const XMATRIX view = g_app.camera.GetViewMatrix();
    const XMATRIX proj = g_app.camera.GetProjMatrix(aspect);

    // HLSL 側で mul(float4(v,1), M) と書けるように転置して格納する
    XMStoreFloat4x4(&fc.View,      XMMatrixTranspose(view));
    XMStoreFloat4x4(&fc.Proj,      XMMatrixTranspose(proj));
    XMStoreFloat4x4(&fc.ViewProj, XMMatrixTranspose(view * proj));

    const XMVECTOR camPos = g_app.camera.GetPosition();
    fc.CameraPosW = XMVECTOR4(camPos.x, camPos.y, camPos.z, 1.0f);

    // 光源: やや手前上方から谷を照らす (光源へ向かう単位ベクトル)
    const XMVECTOR lightW = XMVector3Normalize(XMVectorSet(-0.35f, 0.85f, 0.45f, 0.0f));
    const XMVECTOR lightV = XMVector3Normalize(XMVector3TransformNormal(lightW, view));
    XMStoreFloat4(&fc.LightDirW, lightW);
    XMStoreFloat4(&fc.LightDirV, lightV);

    fc.ScreenSize = XMVECTOR4(
        static_cast<float>(gfx.GetWidth()), static_cast<float>(gfx.GetHeight()),
        1.0f / gfx.GetWidth(), 1.0f / gfx.GetHeight());

    fc.Params = XMVECTOR4(0.15f, g_app.sph.GetSimTime(), 0.0f, 0.0f);
}

```

```

gfx.UpdateBuffer(g_app.cbFrame.Get(), &fc, sizeof(fc));

ID3D11Buffer* cb = g_app.cbFrame.Get();
gfx.GetContext()->VSSetConstantBuffers(0, 1, &cb);
gfx.GetContext()->PSSetConstantBuffers(0, 1, &cb);
}

```

3.4 頂点バッファを使わない描画

頂点シェーダーには、頂点バッファの代わりに「何番目の頂点か」(`SV_VertexID`) と「何番目のインスタンスか」(`SV_InstanceID`) を受け取る方法があります。座標を番号から計算できるなら、頂点バッファは不要です。本プログラムはこれを 2 か所で使います。

全画面三角形

後処理（ぼかしや合成）では画面全体を覆う面を描きます。四角形（三角形 2 枚）でもよいのですが、大きな三角形 1 枚のほうが対角線上でピクセルが 2 回処理される無駄がなく、頂点バッファも要りません。

shaders/Common.hlsl

```

//-----
// 関数名 : VSFullscreen
// 概要   : 頂点バッファなしで画面全体を覆う 1 枚の三角形を生成する
//         vid=0: (-1, 1), vid=1: (3, 1), vid=2: (-1, -3) → 画面全体を覆う
// 引数   : vid : SV_VertexID (0, 1, 2)
// 戻り値 : FullscreenVSOut
//-----
FullscreenVSOut VSFullscreen(uint vid : SV_VertexID)
{
    FullscreenVSOut o;
    float2 uv = float2((vid << 1) & 2, vid & 2);          // (0,0), (2,0), (0,2)
    o.posH = float4(uv * float2(2.0, -2.0) + float2(-1.0, 1.0), 0.0, 1.0);
    o.uv = uv;
    return o;
}

```

vid = 0, 1, 2 に対して uv は (0,0), (2,0), (0,2) となり、クリップ座標は (-1,1), (3,1), (-1,-3)。画面 (-1~1) の外にはみ出した部分はクリップされ、画面全体がちょうど覆われます。

インスタンスング

同じ形状を何千個も描くとき、1 個ずつ描画命令を出すと CPU がボトルネックになります。DrawInstanced(頂点数, インスタンス数) は 1 回の命令で N 個描き、頂点シェーダーは `SV_InstanceID` で「今どの個体か」を知ります。粒子の描画（第 14 章）では、粒子の位置を `StructuredBuffer` に入れておき、`Particles[iid]` で読み出します。

3.5 シェーダーのコンパイル

HLSL は実行前にバイトコードへコンパイルします。本プログラムは起動時に `D3DCompileFromFile` でコンパイルするため、HLSL を書き換えて再起動するだけで結果を確認できます（C++ の再ビルドは不要）。コンパイルエラーはメッセージボックスに表示されます。

```
//-----
// CompileShaderFromFile
//   D3DCompileFromFile を呼び、失敗時はエラーメッセージを表示する。
//-----
bool Graphics::CompileShaderFromFile(const std::wstring& path, const char* entryPoint,
                                     const char* target, ComPtr<ID3DBlob>& outBlob)
{
    UINT flags = D3DCOMPILE_ENABLE_STRICTNESS;
#ifdef _DEBUG
    flags |= D3DCOMPILE_DEBUG | D3DCOMPILE_SKIP_OPTIMIZATION;
#else
    flags |= D3DCOMPILE_OPTIMIZATION_LEVEL3;
#endif

    ComPtr<ID3DBlob> errorBlob;
    HRESULT hr = D3DCompileFromFile(
        path.c_str(),
        nullptr,                      // マクロ定義なし
        D3D_COMPILE_STANDARD_FILE_INCLUDE, // #include を許可
        entryPoint,
        target,
        flags,
        0,
        outBlob.ReleaseAndGetAddressOf(),
        errorBlob.GetAddressOf());

    if (FAILED(hr))
    {
        std::string msg = "シェーダーのコンパイルに失敗しました:\n";
        if (errorBlob)
            msg += static_cast<const char*>(errorBlob->GetBufferPointer());
        else
            msg += "(ファイルが見つからない可能性があります)";

        // UTF-8 → ワイド文字に変換してメッセージボックスで表示
        int len = MultiByteToWideChar(CP_UTF8, 0, msg.c_str(), -1, nullptr, 0);
        std::wstring wmsg(len, L'\0');
        MultiByteToWideChar(CP_UTF8, 0, msg.c_str(), -1, &wmsg[0], len);
        MessageBoxW(nullptr, (wmsg + L"\n" + path).c_str(), L"HLSL エラー", MB_OK | MB_ICONERROR);
        return false;
    }
    return true;
}
```

演習 3

1. `Terrain.hlsl` の `PSMain` の最後を `return float4(n * 0.5 + 0.5, 1);` に変えて、法線を色として表示してみましょう（デバッグでよく使う技法です）。
2. `UpdateFrameConstants` の `XMMatrixTranspose` を外すと何が起きるか試し、理由を説明してください。
3. `VSFullscreen` の 3 頂点が画面全体を覆うことを、紙に $-1 \sim 1$ の正方形と三角形を描いて確かめてください。

ライティングとテクスチャ

同じ形でも、光の当たり方と表面の模様で見え方はまったく変わります。この章では、面の向きと光の方向から明るさを求める「ライティング」の基本モデルと、画像を面に貼る「テクスチャマッピング」を、地形と水面の描画コードで学びます。

4.1 法線 — 面の向き

ライティングの出発点は、面がどちらを向いているかを表す単位ベクトル、**法線 N** です。三角形の法線は 2 辺の外積で求めますが、滑らかな曲面に見せたい場合は、頂点ごとに周囲の面の法線を平均した「頂点法線」を持たせ、ラスタライザに補間させます。高さマップ地形では、高さの偏微分から直接求めることができます（第 6 章）。

4.2 ランバート拡散反射

ざらざらした面は光をあらゆる方向に均等に散らします（拡散反射）。その明るさは、面に光が「どれだけ正面から当たるか」で決まり、法線 **N** と光源へ向かう方向 **L** の内積で表せます。

$$I_{\text{diffuse}} = \max(0, \mathbf{N} \cdot \mathbf{L}) = \max(0, \cos \theta)$$



図 4-1 ランバート拡散反射

$\max(0, \dots)$ は、光が裏側から当たる場合（内積が負）を 0 にするためです。実際にはこれに**環境光**（あらゆる方向から回り込む光の近似）を足し、真っ暗な部分を作らないようにします。地形シェーダーでは、上を向いた面ほど空からの光を受けるという「半球ライティング」風の環境光を使っています。

shaders/Terrain.hlsl

```
//-----
// 関数名 : PSMain
// 概要   : テクスチャ色 × (環境光 + 拡散反射) で色を決める。
//          さらにカメラから遠い谷底ほど少し暗くして奥行き感を出す。
// 引数   : i : 補間済み頂点データ
// 戻り値 : 最終色 (RGBA)
//-----
float4 PSMain(PSIn i) : SV_Target
{
    float3 n = normalize(i.nrm);
    float3 tex = RockTex.Sample(LinearWrap, i.uv).rgb;

    // 2 つ目のテクスチャサンプル（異なるスケール）を掛けて繰り返し感を減らす
```

```

float3 tex2 = RockTex.Sample(LinearWrap, i.uv * 0.23 + 0.31).rgb;
tex = tex * (0.75 + 0.5 * tex2);

// ランバート拡散反射: max(0, N · L)
float diff = saturate(dot(n, LightDirW.xyz));

// 半球ライティング風の環境光: 上を向いた面ほど明るい
float ambient = 0.30 + 0.15 * saturate(n.y);

float3 col = tex * (ambient + 0.75 * diff);

// 谷底 (低い場所) を少し暗くしてくぼみを強調
float depthShade = saturate((i.posW.y + 0.20 * i.posW.z + 3.0) / 10.0); // 谷底で 0.3 付近、山で 1
col *= lerp(0.80, 1.0, depthShade);

return float4(col, 1.0);
}

```

4.3 鏡面反射 — Blinn-Phong

つやのある面には、光源の像がハイライトとして映ります。視線方向 $\mathbf{V}$ と光源方向 $\mathbf{L}$ のちょうど中間の向き $\mathbf{H}$ （ハーフベクトル）を法線がどれだけ向いているかで、ハイライトの強さを近似するのが Blinn-Phong モデルです。

$$\mathbf{H} = \text{normalize}(\mathbf{L} + \mathbf{V}), \quad I_{\text{spec}} = \max(0, \mathbf{N} \cdot \mathbf{H})^n$$

指数 n が大きいほどハイライトは小さく鋭くなります（粒子: 40、水面: 48）。

4.4 フレネル反射

水面を斜めから見ると鏡のように空を映し、真上から見ると透けて見えます。この「見る角度で反射率が変わる」現象がフレネル反射で、Schlick の近似式で簡単に計算できます。

$$F = F_0 + (1 - F_0)(1 - \mathbf{N} \cdot \mathbf{V})^5$$

F_0 は正面から見たときの反射率（水は約 0.02～0.06）です。第 15 章の水面シェーダーでは指数 4、 $F_0 = 0.06$ とし、フレネル項の分だけ空の色を混ぜています。

4.5 テクスチャマッピング

面に画像を貼るには、各頂点に「画像のどの位置に対応するか」を示すテクスチャ座標（UV、0～1）を持たせます。ラスタライザが UV を補間し、ピクセルシェーダーで `Texture.Sample(sampler, uv)` と書けば、その位置の色が取れます。

サンプラーの設定

設定	意味	本プログラム
アドレスモード	UV が 0~1 を外れたときの扱い。WRAP は繰り返し、CLAMP は端で固定	地形: WRAP、深度テクスチャ: CLAMP
フィルタ	ピクセルの中心が画素の間にあるときの補間。POINT は最近傍、LINEAR は線形補間、ANISOTROPIC は斜めから見た面の補間を改善	地形: 異方性 8、深度: POINT
ミップマップ	画像を 1/2, 1/4, ... と縮小した系列を用意し、遠くの面には小さい画像を使う。縮小時のちらつき（エイリアシング）を防ぐ	<code>GenerateMips</code> で自動生成

コラム: なぜミップマップが必要か 遠くの地形では、1 ピクセルの中にテクスチャの何十画素分もが収まります。そのうち 1 点だけを拾うと、カメラが少し動くたびに拾う画素が変わってちらつきます。あらかじめ縮小して平均した画像（ミップ）を使えば、1 ピクセルに対応する範囲の平均色が得られます。GPU は UV の変化率から適切なミップレベルを自動で選びます。

繰り返し感の低減

1 枚のテクスチャを WRAP で敷き詰めると、同じ模様の繰り返しが目につきます。地形シェーダーでは、別スケール・別オフセットでもう一度サンプルして乗算し、周期を崩しています。

演習 4

1. `main.cpp` の光源方向 `lightW` を変えて、谷の影の付き方を観察してください。真上 (0,1,0) にすると平坦に見えるのはなぜでしょう。
2. `Terrain.hls1` に Blinn-Phong の鏡面反射を追加して、濡れた岩のような質感にしてみましょう（指数 20~60 で試す）。
3. 地形の UV スケール（`Terrain.cpp` の `uvScale`）を 0.05 と 0.3 にして、テクスチャの繰り返し感と細かさのトレードオフを比べてください。

第 II 部

地形編

乱数から「自然に見える形」を作るノイズの技術、
高さマップによる地形、テクスチャ、そして Blender からのモデル取り込み。

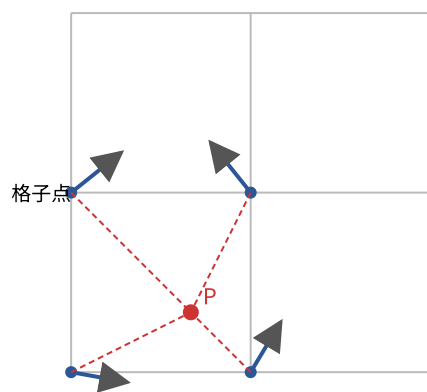
ノイズと手続き的生成

山や岩、雲のような自然物は「規則的でもなく、完全にランダムでもない」形をしています。この章では、そのような形をプログラムで作るための基本部品、パーリンノイズとその重ね合わせ（fBm）を、数式と実装の両面から学びます。

5.1 乱数ではダメな理由

地形の高さを各点で乱数にすると、隣り合う点の高さが無関係になり、砂嵐のような画像になります。自然な起伏に必要なのは「近くの点は似た値、遠くの点は無関係」という**連続性のあるランダム**です。これを作るのが**ノイズ関数**で、CG では Ken Perlin が 1985 年に考案し 2002 年に改良した**パーリンノイズ**が標準的に使われます。

5.2 パーリンノイズの仕組み



1 点 P の値の求め方

- ① P を囲む 4 格子点を求める
- ② 各格子点に固定の勾配ベクトル (青矢印) をハッシュで割り当てる
- ③ 格子点→P のベクトル (赤点線) と勾配の内積を 4 つ計算
- ④ 4 つの値を Fade 曲線で双線形補間

格子点上では内積が 0 なので値は 0、格子の間で山や谷ができる

図 5-1 2 次元パーリンノイズ

格子点ごとの勾配ベクトルは、座標から**置換テーブル**（0～255 をシャッフルした配列）を引いて決めます。同じ座標には常に同じ勾配が対応するので、乱数と違って何度評価しても同じ値になります（再現性）。

Fade 曲線

4 つの内積を単純な線形補間でつなぐと、格子の境界で折れ目が見えます。Perlin の改良版では、両端で 1 階・2 階微分が 0 になる 5 次曲線で補間係数を緩和します。

$$\text{fade}(t) = 6t^5 - 15t^4 + 10t^3$$

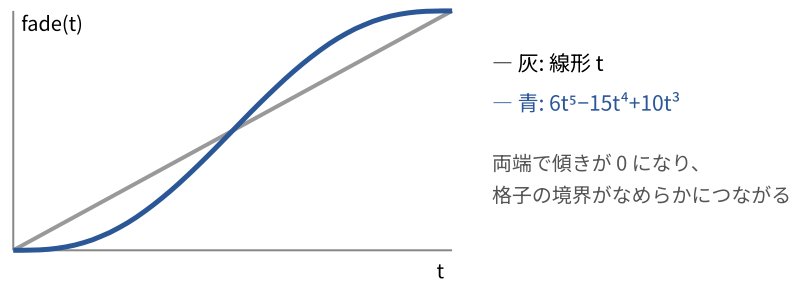


図 5-2 Fade 曲線

src/Noise.cpp

```
//-----
// Perlin2D
// アルゴリズム:
//   1. 評価点を囲む 4 つの格子点 (X,Y), (X+1,Y), (X,Y+1), (X+1,Y+1) を求める
//   2. 各格子点のハッシュ値から勾配を決め、評価点との内積を取る
//   3. 4 つの内積を Fade 曲線で双線形補間する
//-----
float Perlin2D(float x, float y)
{
    // 評価点を含む格子セルの左下の整数座標 (0~255 に丸める)
    const int X = static_cast<int>(std::floor(x)) & 255;
    const int Y = static_cast<int>(std::floor(y)) & 255;

    // セル内での相対位置 (0~1)
    const float xf = x - std::floor(x);
    const float yf = y - std::floor(y);

    // 補間係数 (Fade 曲線で緩和)
    const float u = Fade(xf);
    const float v = Fade(yf);

    // 4 隅のハッシュ値
    const int aa = s_perm[s_perm[X] + Y];
    const int ab = s_perm[s_perm[X] + Y + 1];
    const int ba = s_perm[s_perm[X + 1] + Y];
    const int bb = s_perm[s_perm[X + 1] + Y + 1];

    // x 方向に補間してから y 方向に補間 (双線形補間)
    const float x1 = Lerp(Grad(aa, xf, yf), Grad(ba, xf - 1.0f, yf), u);
    const float x2 = Lerp(Grad(ab, xf, yf - 1.0f), Grad(bb, xf - 1.0f, yf - 1.0f), u);
    return Lerp(x1, x2, v);
}
```

5.3 fBm — 大小の起伏を重ねる

パーリンノイズ 1 層は「なめらかな丘」程度の起伏しか作れません。周波数を 2 倍、振幅を 1/2 にしたノイズを何層も足すと、大きなうねりの上に細かい凹凸が乗った、自然な地形や雲の形になります。これを fBm (fractional Brownian motion) と呼び、各層を**オクターブ**と呼びます。

$$\text{fBm}(x) = \sum_{i=0}^{n-1} g^i \cdot \text{noise}(l^i x) / \sum g^i \quad (l = \text{lacunarity} \approx 2, g = \text{gain} \approx 0.5)$$

オクターブ 0 (周波数 1, 振幅 1)



オクターブ 1 (周波数 2, 振幅 1/2)



オクターブ 2 (周波数 4, 振幅 1/4)



合計 (fBm)



図 5-3 fBm: 周波数を 2 倍、振幅を 1/2 にしたノイズを重ねる (1 次元で図示)

src/Noise.cpp

```
//-----  
// Fbm2D  
// アルゴリズム:  
//   sum = Σ amplitude_i * Perlin2D(x * frequency_i, y * frequency_i)  
//   frequency_i = lacunarity^i, amplitude_i = gain^i  
//   最後に振幅の合計で割って -1~1 に正規化する  
//-----  
float Fbm2D(float x, float y, int octaves, float lacunarity, float gain)  
{  
    float sum = 0.0f;          // ノイズの累積値  
    float amplitude = 1.0f;    // 現在のオクターブの振幅  
    float frequency = 1.0f;    // 現在のオクターブの周波数  
    float norm = 0.0f;         // 正規化用: 振幅の合計  
  
    for (int i = 0; i < octaves; ++i)  
    {  
        sum += amplitude * Perlin2D(x * frequency, y * frequency);  
        norm += amplitude;  
        amplitude *= gain;  
        frequency *= lacunarity;  
    }  
    return sum / norm;  
}
```

5.4 尾根状ノイズ

ノイズの絶対値を取って 1 から引く ($1 - |\text{noise}|$) と、ノイズが 0 を横切る場所に鋭い「尾根」が現れます。これを重ねた尾根状ノイズ (ridged multifractal) は、山の稜線や岩の亀裂の表現に向いています。本プログラムでは 2 乗して尾根をさらに細くしています。

src/Noise.cpp

```
//-----  
// Ridged2D  
// アルゴリズム:
```

```
// 各オクターブで  $r = 1 - |\text{Perlin2D}|$  を計算する。
// ノイズが 0 を横切る場所で  $r$  が 1 になり、鋭い「尾根」が現れる。
//  $r^2$  にすることで尾根をさらに細く鋭くしている。
//-----
float Ridged2D(float x, float y, int octaves)
{
    float sum = 0.0f;
    float amplitude = 1.0f;
    float frequency = 1.0f;
    float norm = 0.0f;

    for (int i = 0; i < octaves; ++i)
    {
        float r = 1.0f - std::fabs(Perlin2D(x * frequency, y * frequency));
        r = r * r; // 尾根を鋭くする
        sum += amplitude * r;
        norm += amplitude;
        amplitude *= 0.5f;
        frequency *= 2.0f;
    }
    return sum / norm;
}
```

5.5 周期化 — 継ぎ目のないテクスチャ

テクスチャは WRAP で敷き詰めるので、右端と左端、上端と下端がつながっていないと継ぎ目が見えます。ノイズの格子座標を周期 P で折り返せば ($X \bmod P$)、格子点 P と 0 が同じ勾配を持ち、値が連続します。

src/Noise.cpp

```
//-----
// Perlin2DPeriodic
// Perlin2D と同じだが、格子の整数座標を period で割った余りにする。
// これにより  $x+1$  が period に達したとき 0 に戻り、端と端がつながる。
//-----
float Perlin2DPeriodic(float x, float y, int period)
{
    const int Xi = static_cast<int>(std::floor(x));
    const int Yi = static_cast<int>(std::floor(y));

    // 負の値でも正しく折り返すように  $((v \% p) + p) \% p$  とする
    auto wrap = [period](int v) { return ((v % period) + period) % period; };
    const int X0 = wrap(Xi) & 255;
    const int Y0 = wrap(Yi) & 255;
    const int X1 = wrap(Xi + 1) & 255;
    const int Y1 = wrap(Yi + 1) & 255;

    const float xf = x - std::floor(x);
    const float yf = y - std::floor(y);
    const float u = Fade(xf);
    const float v = Fade(yf);

    const int aa = s_perm[s_perm[X0] + Y0];
    const int ab = s_perm[s_perm[X0] + Y1];
    const int ba = s_perm[s_perm[X1] + Y0];
    const int bb = s_perm[s_perm[X1] + Y1];

    const float x1 = Lerp(Grad(aa, xf, yf), Grad(ba, xf - 1.0f, yf), u);
    const float x2 = Lerp(Grad(ab, xf, yf - 1.0f), Grad(bb, xf - 1.0f, yf - 1.0f), u);
}
```

```
return Lerp(x1, x2, v);  
}
```

ポイント「乱数の代わりにハッシュを使う」のがノイズの本質です。座標から決定的に値が決まるので、GPU で並列に評価しても、後から同じ場所を評価しても、常に同じ結果が得られます。

演習 5

1. Fbm2D のオクターブ数を 1, 3, 6 と変えて地形を生成し、見た目の違いを比べてください。
2. gain を 0.7 にすると細かい凹凸が強調されます。逆に 0.3 ではどうなるか予想してから試してください。
3. Fade 曲線を `return t;`（線形）に変えて、格子の折れ目が見えるか確認しましょう。

高さマップ地形の設計と生成

地形をどう表現し、どう「蛇行する谷のある山」にデザインし、どうやって GPU が描ける三角形の集まりにするか。この章では高さマップという表現と、そこからメッシュ・法線・テクスチャ座標を作る手順を学びます。作った高さマップは、第 12 章で粒子の衝突判定にも使います。

6.1 高さマップという表現

地形のように「どの (x, z) にも高さが 1 つ決まる」形は、2 次元配列 $h[ix][iz]$ で表せます。これが**高さマップ**です。メモリが少なく、任意の点の高さを補間で即座に取れ、法線も簡単に求まります。欠点は、ひさし状の張り出し（オーバーハング）や洞窟を表せないことです。

本プログラムでは 257×257 の格子で $60 \text{ m} \times 60 \text{ m}$ を覆います（1 セル $\approx 0.23 \text{ m}$ ）。格子点 (ix, iz) のワールド座標は $x = -30 + ix \cdot 0.234$ のように線形に対応します。

6.2 谷のある山をデザインする

「蛇行する谷を水が流れ落ちる」ために、高さ関数を部品に分けて設計します。

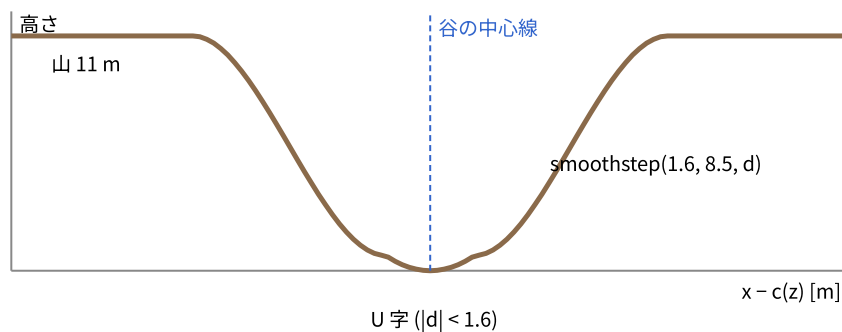


図 6-1 谷の断面（ノイズを除いた基本形）。中心から離れるほど smoothstep で山が立ち上がる

部品	式	役割
谷の中心線	$c(z) = 4 \sin(0.16z) + 2 \sin(0.41z + 1.7)$	周期の違う 2 つの正弦波で自然な蛇行
壁の立ち上がり	$w = \text{smoothstep}(1.6, 8.5, x - c)$	谷底 (0) から山 (1) へ滑らかに遷移
傾斜	$-0.20 z$	上流 ($-z$) が高く、水が流れる
谷底の断面	$0.3 \cdot \min(d, 1.6)^2$	浅い U 字で水を中央に集める
山	$11 w$	両側に高さ 11 m の山
凹凸	$\text{fBm} \cdot (0.25 + 5w)$	谷底は小さく、山肌は大きく
稜線	$\text{ridged} \cdot 1.5 w$	岩の筋

src/Terrain.cpp

```
//-----
// HeightFunction
// アルゴリズム:
//   d      = |x - 谷の中心 x|          ... 谷の中心からの水平距離
//   wallT   = smoothstep(1.6, 8.5, d)   ... 0 = 谷底, 1 = 山の斜面
//   base    = -0.20 * z                 ... 上流 (z=-30) が高く下流 (z=+30) が低い傾斜
//   floorU  = min(d,1.6)^2 * 0.3        ... 谷底を浅い U 字断面にして水を中央へ集める
//   mountain = 11.0 * wallT             ... 両側の山の高さ
//   rough   = fBm * (0.25 + 5.0 * wallT) ... 谷底は小さく、山肌は大きく凹凸を付ける
//   ridge   = ridged * 1.5 * wallT      ... 山肌の稜線状の岩の筋
//   h       = base + floorU + mountain + rough + ridge
//-----
float Terrain::HeightFunction(float x, float z)
{
    const float cx    = ValleyCenterX(z);
    const float d     = std::fabs(x - cx);
    const float wallT = SmoothStep(1.6f, 8.5f, d);

    const float base    = -0.20f * z;
    const float dFloor  = std::min(d, 1.6f);
    const float floorU  = dFloor * dFloor * 0.3f;
    const float mountain = 11.0f * wallT;

    const float fbm     = Noise::Fbm2D(x * 0.05f + 7.3f, z * 0.05f + 2.1f, 6);
    const float rough   = fbm * (0.25f + 5.0f * wallT);

    const float ridge = Noise::Ridged2D(x * 0.12f + 3.0f, z * 0.12f + 5.0f, 4) * 1.5f * wallT;

    return base + floorU + mountain + rough + ridge;
}
```

コラム: smoothstep $\text{smoothstep}(a, b, x)$ は $x \leq a$ で 0、 $x \geq b$ で 1、間を $3t^2 - 2t^3$ でつなぐ関数です。「ここからここまでは 0 から 1 に変わる」を折れ目なく表せるため、CG では閾値処理・マスク・遷移のあらゆる場面に登場します。HLSL には組み込み関数があります。

6.3 高さの補間と法線

粒子の衝突判定では、格子点の間の任意の位置の高さが必要です。周囲 4 点を**双線形補間**します。

src/Terrain.cpp

```
//-----  
// GetHeight  
// アルゴリズム (双線形補間):  
// 1. ワールド座標 → 格子座標 (fx, fz) に変換  
// 2. 整数部 (ix, iz) と小数部 (tx, tz) に分ける  
// 3. 4 隅の高さ h00, h10, h01, h11 を tx, tz で補間  
//-----  
float Terrain::GetHeight(float x, float z) const  
{  
    // 格子座標へ変換し、範囲内にクランプ  
    float fx = (x + HALF_SIZE) / m_cellSize;  
    float fz = (z + HALF_SIZE) / m_cellSize;  
    fx = std::clamp(fx, 0.0f, static_cast<float>(GRID_N - 1) - 0.001f);  
    fz = std::clamp(fz, 0.0f, static_cast<float>(GRID_N - 1) - 0.001f);  
  
    const int ix = static_cast<int>(fx);  
    const int iz = static_cast<int>(fz);  
    const float tx = fx - ix;  
    const float tz = fz - iz;  
  
    const float h00 = m_heights[static_cast<size_t>(iz) * GRID_N + ix];  
    const float h10 = m_heights[static_cast<size_t>(iz) * GRID_N + ix + 1];  
    const float h01 = m_heights[static_cast<size_t>(iz + 1) * GRID_N + ix];  
    const float h11 = m_heights[static_cast<size_t>(iz + 1) * GRID_N + ix + 1];  
  
    const float h0 = h00 + (h10 - h00) * tx; // 下側の辺を x 方向に補間  
    const float h1 = h01 + (h11 - h01) * tx; // 上側の辺を x 方向に補間  
    return h0 + (h1 - h0) * tz; // z 方向に補間  
}
```

法線は、高さ関数 $y = h(x, z)$ の偏微分から求められます。曲面 $h(x, z) - y = 0$ の勾配 $(\partial h / \partial x, -1, \partial h / \partial z)$ は面に垂直なので、上向きに符号を直して正規化します。偏微分は**中心差分** (両隣の差 ÷ 距離) で近似します。

$$\mathbf{N} = \text{normalize}(-\partial h / \partial x, 1, -\partial h / \partial z), \quad \partial h / \partial x \approx (h(x+e, z) - h(x-e, z)) / 2e$$

src/Terrain.cpp

```
//-----  
// GetNormal  
// 中心差分: hx = (h(x+e) - h(x-e)) / 2e, hz = (h(z+e) - h(z-e)) / 2e  
// 曲面 y = h(x, z) の法線は (-hx, 1, -hz) を正規化したもの  
//-----  
XMVECTOR Terrain::GetNormal(float x, float z) const  
{  
    const float e = m_cellSize;  
    const float hx = (GetHeight(x + e, z) - GetHeight(x - e, z)) / (2.0f * e);  
    const float hz = (GetHeight(x, z + e) - GetHeight(x, z - e)) / (2.0f * e);  
  
    XMVECTOR n = XMVector3Normalize(XMVectorSet(-hx, 1.0f, -hz, 0.0f));  
    XMVECTOR out;  
    XMStoreFloat3(&out, n);  
    return out;  
}
```

6.4 メッシュの生成

GPU は三角形しか描けないので、格子を三角形に分割します。各セルを対角線で 2 枚に分け、頂点は格子点をそのまま使い、**インデックスバッファ**で三角形の頂点番号を並べます（頂点を共有するのでメモリが約 1/6 で済みます）。

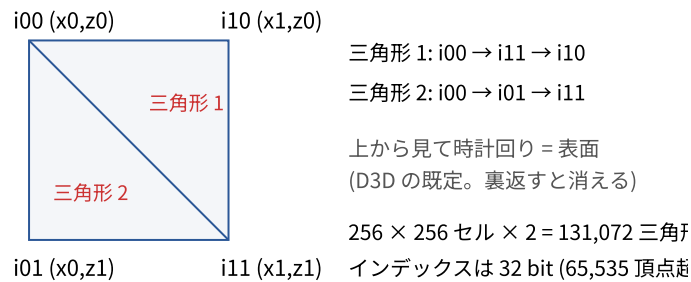


図 6-2 セルの三角形分割と巻き順

src/Terrain.cpp

```
//-----  
// BuildMesh  
// アルゴリズム:  
// 頂点 : 格子点ごとに位置・法線・UV を作る (UV はワールド座標に比例させてタイリング)  
// 索引 : 各セルを 2 枚の三角形に分割する。  
// D3D の既定では「画面上で時計回り」が表面なので、  
// 上から見下ろしたときに時計回りになる順序で並べる。  
//-----  
bool Terrain::BuildMesh(Graphics& gfx)  
{  
    // ---- 頂点 ----  
    std::vector<Vertex> vertices(static_cast<size_t>(GRID_N) * GRID_N);  
    const float uvScale = 0.12f;    // ワールド 1 単位あたりの UV 進み (約 8.3 単位でテクスチャ 1 周)  
  
    for (int iz = 0; iz < GRID_N; ++iz)  
    {  
        const float z = -HALF_SIZE + iz * m_cellSize;  
        for (int ix = 0; ix < GRID_N; ++ix)  
        {  
            const float x = -HALF_SIZE + ix * m_cellSize;  
            Vertex& v = vertices[static_cast<size_t>(iz) * GRID_N + ix];  
            v.pos = XMFLOAT3(x, m_heights[static_cast<size_t>(iz) * GRID_N + ix], z);  
            v.normal = GetNormal(x, z);  
            v.uv = XMFLOAT2(x * uvScale, z * uvScale);  
        }  
    }  
  
    // ---- インデックス ----  
    std::vector<uint32_t> indices;  
    indices.reserve(static_cast<size_t>(GRID_N - 1) * (GRID_N - 1) * 6);  
    for (int iz = 0; iz < GRID_N - 1; ++iz)  
    {  
        for (int ix = 0; ix < GRID_N - 1; ++ix)  
        {  
            const uint32_t i00 = static_cast<uint32_t>(iz * GRID_N + ix);    // (x0, z0)  
            const uint32_t i10 = i00 + 1;    // (x1, z0)  
            const uint32_t i01 = i00 + GRID_N;    // (x0, z1)  
            const uint32_t i11 = i01 + 1;    // (x1, z1)  
  
            // 三角形 1: (x0,z0) → (x1,z1) → (x1,z0)
```

```

        indices.push_back(i00); indices.push_back(i11); indices.push_back(i10);
        // 三角形 2: (x0,z0) → (x0,z1) → (x1,z1)
        indices.push_back(i00); indices.push_back(i01); indices.push_back(i11);
    }
}
return CreateGpuBuffers(gfx, vertices, indices);
}

```

テクスチャ座標はワールド座標に比例させ $(x, z) \times 0.12$ としています。約 8.3 m ごとに画像が 1 周し、WRAP で敷き詰められます。

巻き順を間違えると 三角形が全部裏面と判定されて消えます。「何も描かれない」ときは、まずカリングを無効にして描かれるか確かめるのが定石です。

演習 6

1. 谷の中心線 `ValleyCenterX` の振幅を 8 にして、蛇行を強くしてみましょう。水が壁に当たって溜まる場所が出来ますか。
2. 傾斜 -0.20 を -0.05 にすると水はどうなりますか。流速と滞留の関係を観察してください。
3. `GRID_N` を 129 に下げると、地形の見た目と衝突の滑らかさはどう変わりますか（頂点数と FPS も比べてください）。

テクスチャの生成と画像ファイルの読み込み

岩肌の模様は、画像ファイルを読み込む方法と、ノイズから計算で作る方法の 2 通りを実装しています。この章では、CPU 上のピクセル配列を GPU テクスチャにする共通の手順、WIC による画像読み込み、そしてノイズによる手続きのテクスチャを学びます。

7.1 ピクセル配列から GPU テクスチャへ

どちらの方法でも最終的には「RGBA 各 8bit のピクセルが幅 × 高さ個並んだ配列」ができ、それをテクスチャにします。ミップマップも同時に作ります。

src/TextureLoader.cpp

```
//-----
// CreateFromPixels
// ミップマップ自動生成のため、RENDER_TARGET バインドと GENERATE_MIPS フラグを付ける。
// MipLevels = 0 にすると 1x1 までの全レベルが確保される。
//-----
bool CreateFromPixels(Graphics& gfx, const void* pixels, UINT width, UINT height,
                    ComPtr<ID3D11ShaderResourceView>& outSRV)
{
    if (!pixels || width == 0 || height == 0)
        return false;

    D3D11_TEXTURE2D_DESC td = {};
    td.Width = width;
    td.Height = height;
    td.MipLevels = 0; // 0 = 全ミップレベル
    td.ArraySize = 1;
    td.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
    td.SampleDesc.Count = 1;
    td.Usage = D3D11_USAGE_DEFAULT;
    td.BindFlags = D3D11_BIND_SHADER_RESOURCE | D3D11_BIND_RENDER_TARGET;
    td.MiscFlags = D3D11_RESOURCE_MISC_GENERATE_MIPS;

    ComPtr<ID3D11Texture2D> tex;
    if (FAILED(gfx.GetDevice()->CreateTexture2D(&td, nullptr, tex.GetAddressOf())))
        return false;

    // レベル 0 にピクセルを書き込む (行ピッチ = 幅 × 4 バイト)
    gfx.GetContext()->UpdateSubresource(tex.Get(), 0, nullptr, pixels, width * 4, 0);

    D3D11_SHADER_RESOURCE_VIEW_DESC srvd = {};
    srvd.Format = td.Format;
    srvd.ViewDimension = D3D11_SRV_DIMENSION_TEXTURE2D;
    srvd.Texture2D.MipLevels = static_cast<UINT>(-1); // 全ミップレベル

    ComPtr<ID3D11ShaderResourceView> srv;
    if (FAILED(gfx.GetDevice()->CreateShaderResourceView(tex.Get(), &srvd, srv.GetAddressOf())))
        return false;

    gfx.GetContext()->GenerateMips(srv.Get());
}
```

```

    outSRV = srv;
    return true;
}

```

- `MipLevels = 0` で「1×1 までの全レベルを確保」、`GENERATE_MIPS` フラグと `RENDER_TARGET` バインドで GPU にミップ生成を任せられます。
- `UpdateSubresource` はレベル 0 だけに書き、`GenerateMips` が残りを縮小して埋めます。

7.2 WIC による画像ファイルの読み込み

Windows には画像コーデック基盤 WIC（Windows Imaging Component）が標準搭載されており、PNG / JPEG / BMP / TIFF / GIF などを追加ライブラリなしで読めます。COM ベースの API なので手順はやや長いですが、流れは単純です。

1. ファクトリ `IWICImagingFactory` を作る（COM 初期化が必要）
2. `CreateDecoderFromFilename` でファイルを開く。形式は拡張子でなく内容で判定される
3. 先頭フレームを取り出す（アニメ GIF などは複数フレーム）
4. `IWICFormatConverter` で 32bpp RGBA に統一（元がパレット・グレー・CMYK でも吸収）
5. `CopyPixels` でメモリに展開し、7.1 の手順でテクスチャ化

src/TextureLoader.cpp

```

//-----
// LoadFromFile
//-----
bool LoadFromFile(Graphics& gfx, const std::wstring& path,
                  ComPtr<ID3D11ShaderResourceView>& outSRV,
                  UINT* outWidth, UINT* outHeight)
{
    IWICImagingFactory* factory = GetWICFactory();
    if (!factory)
        return false;

    // 1. デコーダでファイルを開く（拡張子ではなく中身で形式を判定してくれる）
    ComPtr<IWICBitmapDecoder> decoder;
    HRESULT hr = factory->CreateDecoderFromFilename(
        path.c_str(), nullptr, GENERIC_READ, WICDecodeMetadataCacheOnDemand, decoder.GetAddressOf());
    if (FAILED(hr))
        return false;

    // 2. 先頭フレーム（静止画は 1 枚、GIF などは複数枚のうち最初）
    ComPtr<IWICBitmapFrameDecode> frame;
    hr = decoder->GetFrame(0, frame.GetAddressOf());
    if (FAILED(hr))
        return false;

    // 3. ピクセル形式を 32bpp RGBA (R が先頭バイト) に統一
    ComPtr<IWICFormatConverter> converter;
    hr = factory->CreateFormatConverter(converter.GetAddressOf());
    if (FAILED(hr))
        return false;
    hr = converter->Initialize(frame.Get(), GUID_WICPixelFormat32bppRGBA,
                              WICBitmapDitherTypeNone, nullptr, 0.0, WICBitmapPaletteTypeCustom);
    if (FAILED(hr))

```

```

        return false;

    UINT width = 0, height = 0;
    hr = converter->GetSize(&width, &height);
    if (FAILED(hr) || width == 0 || height == 0)
        return false;

    // 4. メモリへ展開
    const UINT stride = width * 4; // 1 行のバイト数
    std::vector<uint8_t> pixels(static_cast<size_t>(stride) * height);
    hr = converter->CopyPixels(nullptr, stride, static_cast<UINT>(pixels.size()), pixels.data());
    if (FAILED(hr))
        return false;

    // 5-6. テクスチャ作成 + ミップマップ生成
    if (!CreateFromPixels(gfx, pixels.data(), width, height, outSRV))
        return false;

    if (outWidth) *outWidth = width;
    if (outHeight) *outHeight = height;
    return true;
}

```

Terrain::CreateRockTexture は assets/rock.png → rock.jpg → ... の順に読み込みを試し、すべて失敗すれば手続き生成に切り替えます。「無ければ落ちる」ではなく「無ければ代替」という設計は、配布時のトラブルを減らします。

7.3 ノイズによる岩テクスチャ

画像がないときは、第 5 章の周期ノイズでピクセルごとに色を決めます。複数の成分を「色の混合」「暗くする」「ざらつかせる」という操作で合成します。

成分	ノイズ	合成の仕方
まだら	fBm (周期 4、6 層)	暗い茶と明るい肌色を lerp
赤斑	fBm (周期 9、別位相)	閾値以上のところに赤を混ぜる
亀裂	尾根ノイズ (周期 10) を smoothstep(0.76, 0.93)	細い線状に暗くする
大きな明暗	fBm (周期 2、3 層)	全体の明るさをゆっくり変える
ざらつき	fBm (周期 64)	±7% の細かい明暗

src/Terrain.cpp

```

//-----
// CreateProceduralRockTexture
// アルゴリズム (ピクセルごと):
//   f1    = タイリング fBm (大きなまだら)    → 明るい肌色と茶色を混ぜる
//   f2    = タイリング fBm (別位相・細かい)    → 赤みのある斑点を加える
//   crack = 尾根状ノイズを閾値処理              → 細い暗い亀裂線
//   fine  = 高周波ノイズ                        → ざらつき
//   color = lerp(dark, light, f1) → 赤斑点を混合 → 亀裂で暗く → ざらつき
//   その後、ミップマップ付きテクスチャを作り GenerateMips で縮小版を自動生成する。
//-----
bool Terrain::CreateProceduralRockTexture(Graphics& gfx)

```

```

{
    const int texSize = 1024;
    std::vector<uint32_t> pixels(static_cast<size_t>(texSize) * texSize);

    const XMFLOAT3 cLight(0.90f, 0.78f, 0.68f);    // 明るいピンクがかった肌色
    const XMFLOAT3 cDark (0.66f, 0.52f, 0.44f);    // やや暗い茶色
    const XMFLOAT3 cRed  (0.74f, 0.42f, 0.34f);    // 赤みのある斑点

    for (int y = 0; y < texSize; ++y)
    {
        const float v = static_cast<float>(y) / texSize;
        for (int x = 0; x < texSize; ++x)
        {
            const float u = static_cast<float>(x) / texSize;

            const float f0 = Noise::FbmTileable(u + 0.9f, v + 0.4f, 2, 3);    // -1~1 (大きな明暗)
            const float f1 = Noise::FbmTileable(u, v, 4, 6);                // -1~1
            const float f2 = Noise::FbmTileable(u + 0.37f, v + 0.11f, 9, 4);    // -1~1
            const float rg = Noise::RidgedTileable(u + 0.5f, v + 0.25f, 10, 3);    // 0~1 (細かい亀裂)
            const float fine = Noise::FbmTileable(u + 0.2f, v + 0.7f, 64, 2);    // -1~1

            // 明暗のまだら
            XMFLOAT3 c = Lerp3(cDark, cLight, 0.5f + 0.5f * f1);
            // 赤みの斑点 (f2 が高い場所だけ)
            c = Lerp3(c, cRed, Saturate(f2 * 1.6f - 0.35f));
            // 亀裂: 尾根ノイズが高い場所を細く暗くする (閾値を高くすると線が細くなる)
            const float crack = SmoothStep(0.76f, 0.93f, rg);
            const float darken = (1.0f - 0.33f * crack) * (0.88f + 0.12f * f0);
            // ざらつき
            const float grain = 0.93f + 0.07f * fine;

            const float r = Saturate(c.x * darken * grain);
            const float g = Saturate(c.y * darken * grain);
            const float b = Saturate(c.z * darken * grain);

            // RGBA8 (リトルエンディアンで R が最下位バイト)
            const uint32_t R = static_cast<uint32_t>(r * 255.0f + 0.5f);
            const uint32_t G = static_cast<uint32_t>(g * 255.0f + 0.5f);
            const uint32_t B = static_cast<uint32_t>(b * 255.0f + 0.5f);
            pixels[static_cast<size_t>(y) * texSize + x] = R | (G << 8) | (B << 16) | (0xFFu << 24);
        }
    }

    // GPU テクスチャ化 (ミップマップ生成込み) はファイル読み込みと共通の処理を使う
    return TextureLoader::CreateFromPixels(gfx, pixels.data(), texSize, texSize, m_rockTexSRV);
}

```

ポイント 手続き的テクスチャは「解像度に依存しない」「パラメータで無限に変えられる」「ファイル不要」という利点があります。一方、写真のような複雑さを出すには多くの層と調整が必要で、実務では写真テクスチャと組み合わせて使うのが普通です。

演習 7

1. `assets/rock.png` を好きな写真（シームレスでなくてもよい）に差し替えて、継ぎ目がどう見えるか観察してください。
2. 手続き生成の亀裂の閾値 0.76 を 0.6 にすると、亀裂はどう変わりますか。
3. `GenerateMips` の呼び出しをコメントアウトし、遠景でのちらつきを確認してください（サンプラーの `MaxLOD` も 0 にする必要があるか考えてみましょう）。

外部メッシュの取り込み — OBJ と Blender

自分で式を書く代わりに、Blender などのツールで作った地形を使いたいこともあります。この章では、最も単純な 3D ファイル形式である Wavefront OBJ の読み方、座標系の変換、そして「メッシュを高さマップに焼き込む」ことで衝突判定を変えずに任意形状を扱う方法を学びます。

8.1 OBJ 形式

OBJ は行ごとに要素を書くテキスト形式です。

```
v -30.0 6.19 30.0      # 頂点座標 (1 番から数える)
vt 0.0 0.0             # テクスチャ座標
vn 0.0 1.0 0.0         # 法線
f 1/1/1 2/2/2 123/123/123 122/122/122  # 面: 頂点/UV/法線 の番号の組 (4 頂点以上も可)
```

面の各頂点は `a`, `a/b`, `a//c`, `a/b/c` の 4 通りの書き方があり、負の数は「末尾から数えて」の相対指定です。Blender は既定で四角形 (quad) を出力するので、三角形化が必要です。

頂点の共有

OBJ では位置・UV・法線が別々の配列で、面の頂点は「その組み合わせ」を指します。GPU の頂点は 1 つの構造体なので、(v, vt, vn) の組ごとに頂点を作り、同じ組が再登場したら再利用します。ハッシュマップで管理します。

src/MeshLoader.cpp

```
//-----
// LoadObj
// アルゴリズム:
//   1. 行ごとに先頭キーワードで分岐し、v / vt / vn を一時配列に貯める
//   2. f 行は各トークンを IndexTriple に変換し、
//      (v, vt, vn) が未登録なら新しい頂点を作ってマップに登録、既存なら再利用
//   3. 多角形は (0, i, i+1) の扇状分割で三角形にする
//-----
bool LoadObj(const std::wstring& path, MeshData& out, std::string* errorMsg)
{
    std::ifstream file(path);    // MSVC は wstring パスを受け付ける
    if (!file)
    {
        if (errorMsg) *errorMsg = "file open failed";
        return false;
    }

    std::vector<XMFLLOAT3> positions;    // v
    std::vector<XMFLLOAT2> texcoords;    // vt
    std::vector<XMFLLOAT3> normals;      // vn
    std::unordered_map<IndexTriple, uint32_t, IndexTripleHash> vertexMap;    // (v,vt,vn) → 出力頂点番号

    out = MeshData();

    std::string line;
```

```

while (std::getline(file, line))
{
    if (line.empty() || line[0] == '#')
        continue;

    std::istringstream ss(line);
    std::string key;
    ss >> key;

    if (key == "v")
    {
        XMFLOAT3 p = { 0, 0, 0 };
        ss >> p.x >> p.y >> p.z;
        positions.push_back(p);
    }
    else if (key == "vt")
    {
        XMFLOAT2 t = { 0, 0 };
        ss >> t.x >> t.y;
        // OBJ の v は下がり 0、D3D のテクスチャ座標は上がり 0 なので反転する
        t.y = 1.0f - t.y;
        texcoords.push_back(t);
    }
    else if (key == "vn")
    {
        XMFLOAT3 n = { 0, 1, 0 };
        ss >> n.x >> n.y >> n.z;
        normals.push_back(n);
    }
    else if (key == "f")
    {
        std::vector<uint32_t> faceIndices;    // この面の出力頂点番号
        std::string token;
        while (ss >> token)
        {
            const IndexTriple t = ParseFaceVertex(token,
                static_cast<int>(positions.size()),
                static_cast<int>(texcoords.size()),
                static_cast<int>(normals.size()));
            if (t.v < 0)
                continue;    // 不正な頂点参照は無視

            auto it = vertexMap.find(t);
            if (it == vertexMap.end())
            {
                MeshVertex mv;
                mv.pos      = positions[t.v];
                mv.normal = (t.vn >= 0) ? normals[t.vn] : XMFLOAT3(0, 1, 0);
                mv.uv       = (t.vt >= 0) ? texcoords[t.vt] : XMFLOAT2(0, 0);
                const uint32_t newIndex = static_cast<uint32_t>(out.vertices.size());
                out.vertices.push_back(mv);
                it = vertexMap.emplace(t, newIndex).first;
                if (t.vn >= 0) out.hasNormals = true;
                if (t.vt >= 0) out.hasUVs = true;
            }
            faceIndices.push_back(it->second);
        }

        // 扇状三角形分割: (0,1,2), (0,2,3), (0,3,4), ...
        for (size_t i = 1; i + 1 < faceIndices.size(); ++i)
        {

```

```

        out.indices.push_back(faceIndices[0]);
        out.indices.push_back(faceIndices[i]);
        out.indices.push_back(faceIndices[i + 1]);
    }
}
// その他 (o, g, s, usemtl, mtl, lib, l ...) は無視
}

if (out.indices.size() < 3)
{
    if (errorMsg) *errorMsg = "no faces";
    return false;
}

if (!out.hasNormals)
    ComputeSmoothNormals(out);
ComputeBounds(out);
return true;
}

```

多角形の三角形化は**扇状分割** (0, i , $i+1$) です。凸な多角形なら常に正しく、Blender の quad はほぼ凸なので実用上十分です。

8.2 座標系の変換

Blender は右手系・Z 上、本プログラムは左手系・Y 上です。変換は 2 段階で行います。

Blender (x, y, z) — エクスポート (Forward -Z, Up Y) → OBJ ($x, z, -y$) — 読み込み時 x 反転 → DirectX ($-x, z, -y$)

1 段目は Blender のエクスポート設定で、Y を上にする軸の入れ替えです。これで右手系 Y-up になります。2 段目で x を反転すると左手系になります（鏡映は右手系と左手系を入れ替えます）。結果として、Blender の上面図で上 (+Y) が奥（上流）、右 (+X) が画面の右になり、Blender で見たままの構図で表示されます。

巻き順の統一

鏡映すると三角形の巻き順が反転します。さらに元データの巻き順は保証されません。そこで三角形ごとに $\mathbf{e}_1 \times \mathbf{e}_2$ の y 成分を調べ、負（下向き）なら頂点 1, 2 を入れ替えます。地形は「上から見える面が表」なので、これで第 6 章のメッシュと同じ規約に揃います。

src/Terrain.cpp

```

//-----
// LoadMeshTerrain
//-----
bool Terrain::LoadMeshTerrain(Graphics& gfx)
{
    MeshLoader::MeshData mesh;
    if (!MeshLoader::LoadObj(Graphics::AssetPath(TERRAIN_MESH_FILE), mesh))
        return false;

    // ---- 1. 右手系 (Blender/OBJ) → 左手系 (DirectX): x を反転 ----
    for (auto& v : mesh.vertices)
    {

```

```

        v.pos.x      = -v.pos.x;
        v.normal.x   = -v.normal.x;
    }
    MeshLoader::ComputeBounds(mesh);

    // ---- 2. 自動フィット: xz の大きい方の辺を WORLD_SIZE に、中心を原点に ----
    const XMFLOAT3& mn = mesh.boundsMin;
    const XMFLOAT3& mx = mesh.boundsMax;
    const float extent = std::max(mx.x - mn.x, mx.z - mn.z);
    if (extent < 1e-6f)
        return false;
    const float scale = (WORLD_SIZE * 0.999f) / extent;    // 端が格子の外に出ないように僅かに縮める
    const XMFLOAT3 center((mn.x + mx.x) * 0.5f, (mn.y + mx.y) * 0.5f, (mn.z + mx.z) * 0.5f);
    for (auto& v : mesh.vertices)
    {
        v.pos.x = (v.pos.x - center.x) * scale;
        v.pos.y = (v.pos.y - center.y) * scale;
        v.pos.z = (v.pos.z - center.z) * scale;
    }

    // ---- 3. 三角形の巻き順を統一 ----
    // 左手系 + D3D 既定 (時計回りが表面) では、上を向く面は cross(e1, e2).y > 0 になる
    // (BuildMesh と同じ規約)。地形は上から見える面が表なので、下向きなら順序を入れ替える。
    for (size_t i = 0; i + 2 < mesh.indices.size(); i += 3)
    {
        const XMFLOAT3& p0 = mesh.vertices[mesh.indices[i]].pos;
        const XMFLOAT3& p1 = mesh.vertices[mesh.indices[i + 1]].pos;
        const XMFLOAT3& p2 = mesh.vertices[mesh.indices[i + 2]].pos;
        const float e1x = p1.x - p0.x, e1z = p1.z - p0.z;
        const float e2x = p2.x - p0.x, e2z = p2.z - p0.z;
        const float crossY = e1z * e2x - e1x * e2z;    // cross(e1, e2).y
        if (crossY < 0.0f)
            std::swap(mesh.indices[i + 1], mesh.indices[i + 2]);
    }

    // ---- 4. 衝突判定用の高さマップに焼き込む ----
    BakeHeightMap(mesh);

    // ---- 5. 描画用頂点 (UV が無ければワールド座標から生成) ----
    std::vector<Vertex> vertices(mesh.vertices.size());
    const float uvScale = 0.12f;
    for (size_t i = 0; i < mesh.vertices.size(); ++i)
    {
        const MeshLoader::MeshVertex& mv = mesh.vertices[i];
        vertices[i].pos      = mv.pos;
        vertices[i].normal   = mv.normal;
        vertices[i].uv       = mesh.hasUVs ? mv.uv : XMFLOAT2(mv.pos.x * uvScale, mv.pos.z * uvScale);
    }
    if (!CreateGpuBuffers(gfx, vertices, mesh.indices))
        return false;

    m_meshFromFile = true;
    return true;
}

```

8.3 自動フィット

SPH のパラメータ（粒子間隔 0.3 m、地形 60 m）に合わせるため、バウンディングボックスから一様スケールを決め、中心を原点に移します。高さも同じ倍率なので、Blender 側では縦横の比だけ意識すれば十分です。

8.4 高さマップへの焼き込み

衝突判定は高さマップで行うので、メッシュを真上から「ラスタライズ」して格子点の高さを求めます。手順は GPU のラスタライズと同じ考え方です。

1. 三角形を xz 平面に投影し、包含する格子範囲を求める
2. 範囲内の各格子点で**重心座標**（辺関数 ÷ 面積）を計算し、3 つとも ≥ 0 なら内側
3. 高さを重心座標で補間し、既存値より高ければ採用（上から見える面）
4. 覆われなかった格子点は、隣接点の平均で反復的に穴埋め

$$w_0 = ((x_1 - p_x)(z_2 - p_z) - (x_2 - p_x)(z_1 - p_z)) / \text{area}, \quad w_1 = \dots, \quad w_2 = 1 - w_0 - w_1$$
$$y = w_0 y_0 + w_1 y_1 + w_2 y_2$$

src/Terrain.cpp

```
//-----  
// BakeHeightMap  
// アルゴリズム（三角形ごと）：  
// 1. xz 平面での包含格子範囲 [ix0, ix1] × [iz0, iz1] を求める  
// 2. 各格子点 (x, z) について辺関数（エッジ関数）で重心座標 (w0, w1, w2) を計算  
// 3. すべて  $\geq 0$ （三角形の内側）なら  $y = w_0 y_0 + w_1 y_1 + w_2 y_2$  を高さ候補にする  
// 4. 候補が既存値より高ければ採用（オーバーハングは上面のみ残る）  
// 穴埋め：覆われなかった格子点は、覆われた 4 近傍の平均で埋める処理を  
// 全て埋まるまで繰り返す（最大 GRID_N 回）。  
//-----  
void Terrain::BakeHeightMap(const MeshLoader::MeshData& mesh)  
{  
    const float UNSET = -1e30f; // 未設定を表す番兵  
    m_heights.assign(static_cast<size_t>(GRID_N) * GRID_N, UNSET);  
  
    // ワールド x → 格子添え字（小数）  
    auto toGrid = [this](float w) { return (w + HALF_SIZE) / m_cellSize; };  
  
    for (size_t t = 0; t + 2 < mesh.indices.size(); t += 3)  
    {  
        const XMFLOAT3& p0 = mesh.vertices[mesh.indices[t]].pos;  
        const XMFLOAT3& p1 = mesh.vertices[mesh.indices[t + 1]].pos;  
        const XMFLOAT3& p2 = mesh.vertices[mesh.indices[t + 2]].pos;  
  
        // 1. 包含格子範囲  
        const float gx0 = toGrid(p0.x), gz0 = toGrid(p0.z);  
        const float gx1 = toGrid(p1.x), gz1 = toGrid(p1.z);  
        const float gx2 = toGrid(p2.x), gz2 = toGrid(p2.z);  
        const int ix0 = std::max(0, static_cast<int>(std::floor(std::min({ gx0, gx1, gx2 }))));  
        const int ix1 = std::min(GRID_N - 1, static_cast<int>(std::ceil(std::max({ gx0, gx1, gx2 }))));  
        const int iz0 = std::max(0, static_cast<int>(std::floor(std::min({ gz0, gz1, gz2 }))));  
        const int iz1 = std::min(GRID_N - 1, static_cast<int>(std::ceil(std::max({ gz0, gz1, gz2 }))));
```

```

// 三角形の xz 面積の 2 倍 (0 なら真横から見た縮退三角形なので無視)
const float area = (gx1 - gx0) * (gz2 - gz0) - (gx2 - gx0) * (gz1 - gz0);
if (std::fabs(area) < 1e-9f)
    continue;
const float invArea = 1.0f / area;

for (int iz = iz0; iz <= iz1; ++iz)
{
    for (int ix = ix0; ix <= ix1; ++ix)
    {
        const float px = static_cast<float>(ix);
        const float pz = static_cast<float>(iz);
        // 2. 重心座標 (辺関数 / 面積)
        const float w0 = ((gx1 - px) * (gz2 - pz) - (gx2 - px) * (gz1 - pz)) * invArea;
        const float w1 = ((gx2 - px) * (gz0 - pz) - (gx0 - px) * (gz2 - pz)) * invArea;
        const float w2 = 1.0f - w0 - w1;
        const float eps = -1e-4f; // 辺上の格子点も含めるための許容誤差
        if (w0 < eps || w1 < eps || w2 < eps)
            continue;
        // 3-4. 高さを補間し、より高ければ採用
        const float y = w0 * p0.y + w1 * p1.y + w2 * p2.y;
        float& h = m_heights[static_cast<size_t>(iz) * GRID_N + ix];
        if (y > h)
            h = y;
    }
}

// 穴埋め (覆われなかった格子点を隣接点の平均で埋める)
std::vector<float> next(m_heights.size());
for (int iter = 0; iter < GRID_N; ++iter)
{
    bool anyUnset = false;
    next = m_heights;
    for (int iz = 0; iz < GRID_N; ++iz)
    {
        for (int ix = 0; ix < GRID_N; ++ix)
        {
            const size_t idx = static_cast<size_t>(iz) * GRID_N + ix;
            if (m_heights[idx] != UNSET)
                continue;
            float sum = 0.0f;
            int count = 0;
            const int nx[4] = { ix - 1, ix + 1, ix, ix };
            const int nz[4] = { iz, iz, iz - 1, iz + 1 };
            for (int k = 0; k < 4; ++k)
            {
                if (nx[k] < 0 || nx[k] >= GRID_N || nz[k] < 0 || nz[k] >= GRID_N)
                    continue;
                const float hn = m_heights[static_cast<size_t>(nz[k]) * GRID_N + nx[k]];
                if (hn != UNSET) { sum += hn; ++count; }
            }
            if (count > 0)
                next[idx] = sum / count;
            else
                anyUnset = true;
        }
    }
}
m_heights.swap(next);
if (!anyUnset)
    break;

```

```

}
// メッシュが空などで最後まで埋まらなかった格子点は 0 にする
for (float& h : m_heights)
    if (h == UNSET) h = 0.0f;
}

```

コラム: 重心座標 三角形の内部の点 P は、3 頂点の「重み付き平均」 $P = w_0A + w_1B + w_2C$ (w の和は 1) で一意に表せます。重みは「 P と対辺が作る小三角形の面積 ÷ 全体の面積」に等しく、すべて 0 以上なら P は内側です。GPU が三角形内で頂点属性を補間するときも、まさにこの重心座標を使っています。

8.5 エミッタ位置の決定

手続き生成では谷の中心線を式で持っていました、任意メッシュでは使えません。そこで「上流端の行で最も低い x 」を高さマップから探す `GetValleyCenterX` を用意し、水の投入位置に使います。

src/Terrain.cpp

```

//-----
// GetValleyCenterX
//-----
float Terrain::GetValleyCenterX(float z) const
{
    if (!m_meshFromFile)
        return ValleyCenterX(z);

    // OBJ 地形: この z の行で最も低い x を探す (端 1 m は除外)
    float bestX = 0.0f;
    float bestH = 1e30f;
    for (float x = -HALF_SIZE + 1.0f; x <= HALF_SIZE - 1.0f; x += m_cellSize)
    {
        const float h = GetHeight(x, z);
        if (h < bestH) { bestH = h; bestX = x; }
    }
    return bestX;
}

```

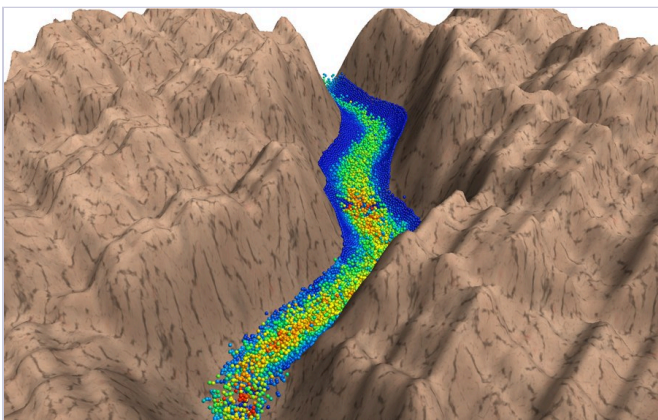


図 8-1 OBJ から取り込んだ試作用の谷 (2 段の滝つき)



図 8-2 同じ地形の表面生成モード

演習 8

1. Blender で平面を細分化し、Proportional Edit で谷を掘って `assets/terrain.obj` に書き出し、水を流してみよう。
2. 読み込み時の x 反転をやめると、Blender の上面図と比べて左右がどうなるか確認してください。
3. 格子の穴埋めを無効化（未設定を 0 のまま）して、メッシュが正方形でないときに何が起きるか観察してください。

第 III 部

流体編

水を「粒子の集まり」として計算する SPH 法。
その考え方から、速く・安定に動かすための実装技術まで。

粒子で流体を表す — 粒子法の考え方

水の動きをコンピュータで計算するには、連続した「流体」を有限個の数値で表す必要があります。この章では、流体シミュレーションの 2 つの流儀（格子法と粒子法）と、粒子法の代表である SPH の基本的な考え方を、数式を最小限にして説明します。

9.1 流体の運動方程式

流体の運動はナビエ・ストークス方程式で記述されます。難しそうな名前ですが、中身はニュートンの運動方程式 $m\mathbf{a} = \mathbf{F}$ を流体に当てはめたものです。

$$\rho (d\mathbf{v}/dt) = -\nabla p + \mu \nabla^2 \mathbf{v} + \rho \mathbf{g}$$

左辺は「密度 × 加速度」、右辺は流体にはたらく 3 つの力です。

項	名前	意味
$-\nabla p$	圧力勾配力	圧力の高い所から低い所へ押される。流体が「詰まりすぎない」ようにする力
$\mu \nabla^2 \mathbf{v}$	粘性力	周囲との速度差を減らす。粘りけ
$\rho \mathbf{g}$	外力（重力）	下向きに引かれる

この式を「どの場所で」計算するかで 2 つの流儀に分かれます。

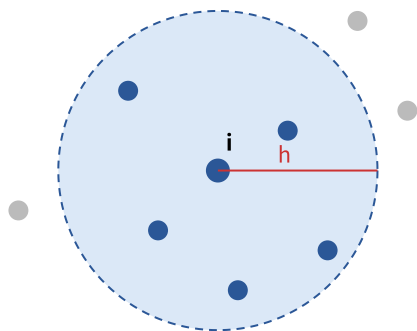
- **格子法（オイラー的）**：空間を格子に切り、各格子で流速や圧力を追う。気体や大規模な水面に向くが、飛沫や自由表面の追跡が難しい。
- **粒子法（ラグランジュ的）**：流体を粒子に分け、粒子が流れに乗って動く。自由表面や飛沫が自然に表現でき、境界の扱いも直感的。本書はこちら。

9.2 SPH の基本アイデア — 「なめらかな粒子」

粒子法の代表 SPH（Smoothed Particle Hydrodynamics）は、もともと天体物理学で星の形成を計算するために生まれ、後に CG に持ち込まれました（Müller ら、2003）。核心は次の 1 つの考え方です。

ポイント 各粒子は「点」ではなく、半径 h の範囲にぼんやり広がった質量の塊とみなす。ある場所の物理量（密度など）は、その場所を覆っている粒子たちの寄与を「距離が近いほど大きい重み」で足し合わせて求める。

半径 h の中の粒子だけが i に影響する



量 A の粒子 i における値

$$A_i = \sum_j (m_j / \rho_j) \cdot A_j \cdot W(r_{ij}, h)$$

m : 質量, ρ : 密度, r_{ij} : 距離

W : カーネル関数 (近いほど大きい重み)

灰色の粒子は h より遠いので無視

→ 「近傍探索」が計算の要 (第 11 章)

図 9-1 SPH の補間。粒子 i の値は、半径 h 内の粒子の重み付き和

この「重み付き和」の式を使えば、密度も圧力の勾配も粘性も、すべて近傍粒子の和として計算できます。微分方程式を解く代わりに「粒子ごとの足し算」に置き換わるのが SPH の魅力です。

9.3 1 ステップの流れ

SPH の時間発展は、毎ステップ次の手順を繰り返します。第 10 章以降で各段を詳しく見ます。

1. 各粒子の近傍 (距離 $< h$) をを見つける (第 11 章)
2. 密度 ρ を計算し、状態方程式から圧力 p を求める
3. 圧力力・粘性力・重力から加速度を求める
4. 速度と位置を更新する (時間積分)
5. 地形・壁との衝突を処理する (第 12 章)

コラム: なぜ「弱圧縮性」なのか 本当の水はほとんど圧縮しません (非圧縮)。しかし非圧縮を厳密に守るには連立方程式を反復で解く必要があり、計算が重くなります。CG では「少しかだけ圧縮を許し、圧縮したら強い圧力で押し返す」という**弱圧縮性 SPH (WCSPH)**がよく使われます。本プログラムもこの方式で、圧縮率は約 0.5% です。厳密な非圧縮手法 (PCISPH, DFSPH) は第 18 章で紹介します。

SPH の理論と実装

前章のアイデアを具体的な数式にし、コードに落とします。カーネル関数の選び方、密度と圧力、力の計算、時間積分、そしてパラメータの決め方まで、実装と 1 対 1 に対応させて解説します。

10.1 カーネル関数

カーネル $W(r, h)$ は「距離 r の粒子の重み」を決める関数で、 $r \geq h$ で 0、全空間で積分すると 1 になるよう正規化されています。Müller らは用途ごとに 3 種類を使い分けました。

$$W_{\text{poly6}}(r) = 315 / (64\pi h^9) \cdot (h^2 - r^2)^3 \quad \dots \text{密度用}$$

$$\nabla W_{\text{spiky}}(r) = -45 / (\pi h^6) \cdot (h - r)^2 \cdot \hat{\mathbf{r}} \quad \dots \text{圧力用}$$

$$\nabla^2 W_{\text{visc}}(r) = 45 / (\pi h^6) \cdot (h - r) \quad \dots \text{粘性用}$$

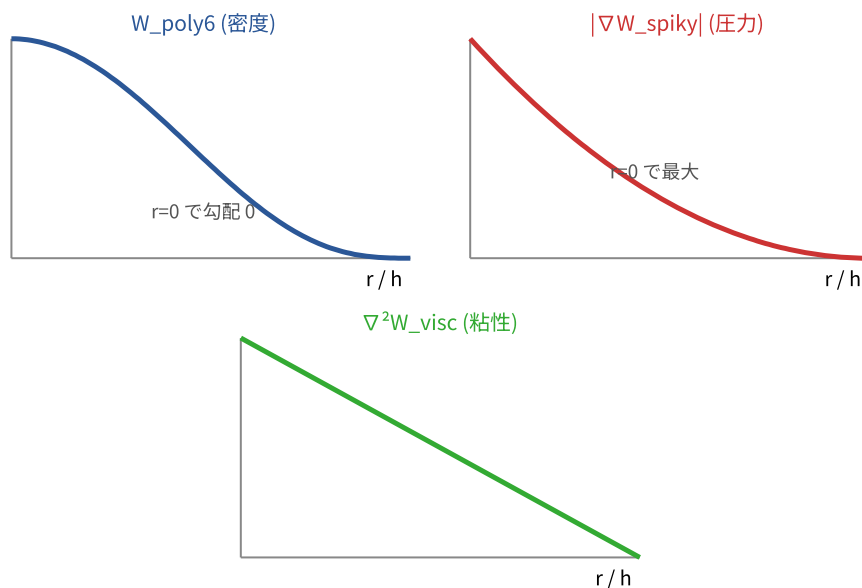


図 10-1 3 種類のカーネル ($h = 1$ で正規化)。poly6 は中心が平ら、spiky は中心で勾配が最大

- poly6 は r^2 だけで計算でき平方根が不要。密度の和に向く。
- poly6 の勾配は $r \rightarrow 0$ で 0 になるため、粒子が重なると押し返す力が消えて凝集してしまう。spiky は中心で勾配が最大なので圧力力に使う。
- visc のラプラシアンは常に正で、粘性力が必ず速度差を減らす向きにはたらく（数値的に安定）。

係数は h から起動時に 1 回だけ計算します。

```
//-----
// Initialize
// カーネル係数 (Müller 2003):
//   W_poly6(r) = 315/(64 π h^9) (h^2 - r^2)^3           (0 ≤ r ≤ h)
//   ∇W_spiky(r) = -45/(π h^6) (h - r)^2 * r̂           (0 < r ≤ h)
//   ∇²W_visc(r) = 45/(π h^6) (h - r)                   (0 ≤ r ≤ h)
//-----
void SPH::Initialize(const Terrain* terrain, const SPHParams& params)
{
    m_terrain = terrain;
    m_params = params;

    m_h = params.smoothingRadius;
    m_h2 = m_h * m_h;
    m_mass = params.restDensity * params.particleSpacing * params.particleSpacing *
params.particleSpacing;

    const float h3 = m_h2 * m_h;
    const float h6 = h3 * h3;
    const float h9 = h6 * h3;
    m_poly6Coef = 315.0f / (64.0f * PI * h9);
    m_spikyGradCoef = 45.0f / (PI * h6);
    m_viscLapCoef = 45.0f / (PI * h6);

    const size_t n = static_cast<size_t>(params.maxParticles);
    m_pos.resize(n);
    m_vel.resize(n);
    m_acc.resize(n);
    m_density.resize(n);
    m_pressure.resize(n);
    m_cell.resize(n);
    m_cellKey.resize(n);
    m_sorted.resize(n);
    m_renderData.reserve(n);

    // ハッシュ表は粒子数の 2 倍以上の 2 のべき乗にして衝突を減らし、剰余をビットマスクで済ませる
    m_tableSize = 1;
    while (m_tableSize < n * 2)
        m_tableSize <= 1;
    m_tableMask = m_tableSize - 1;
    m_cellStart.resize(m_tableSize + 1);
    m_cellFill.resize(m_tableSize);

    m_neighbors.resize(n * static_cast<size_t>(params.maxNeighbors));
    m_neighborCount.resize(n, 0);
    m_speed.resize(n);

    Reset();
}
```

10.2 密度と圧力

密度は「質量の重み付き和」です。自分自身 ($r = 0$) も含めます。

$$\rho_i = \sum_j m_j W_{\text{poly6}}(r_{ij}), \quad p_i = k \cdot \max(\rho_i - \rho_0, 0)$$

圧力は**状態方程式**で密度から直接決めます。静止密度 ρ_0 （水なら 1000 kg/m^3 ）より詰まっていれば正の圧力が生じ、粒子同士を押し離します。負の圧力（引力）を許すと粒子が塊になって不自然になるので、0 で打ち切ります。

src/SPH.cpp

```
//-----  
// ComputeDensityPressure  
//   $\rho_i = \sum_j m W_{\text{poly6}}(r_{ij})$   ( $j = i$  自身も含む)  
//   $p_i = k * \max(\rho_i - \rho_0, 0)$   
//    負の圧力（引力）を許すと粒子が塊になって不自然になるため 0 で打ち切る。  
//-----  
void SPH::ComputeDensityPressure()  
{  
    const int n = m_count;  
    const int maxN = m_params.maxNeighbors;  
    const float mass = m_mass;  
    const float poly6 = m_poly6Coef;  
    const float h2 = m_h2;  
    const float k = m_params.stiffness;  
    const float rho0 = m_params.restDensity;  
  
    #pragma omp parallel for schedule(dynamic, 64)  
    for (int i = 0; i < n; ++i)  
    {  
        const XMFLOAT3 pi = m_pos[i];  
        const int* list = &m_neighbors[static_cast<size_t>(i) * maxN];  
        const int count = m_neighborCount[i];  
  
        float rho = 0.0f;  
        for (int k2 = 0; k2 < count; ++k2)  
        {  
            const XMFLOAT3& pj = m_pos[list[k2]];  
            const float rx = pi.x - pj.x, ry = pi.y - pj.y, rz = pi.z - pj.z;  
            const float r2 = rx * rx + ry * ry + rz * rz;  
            const float d = h2 - r2;    // 近傍リストは  $r2 < h2$  を保証しているので  $d > 0$   
            rho += mass * poly6 * d * d * d;  
        }  
        m_density[i] = rho;  
        m_pressure[i] = k * std::max(rho - rho0, 0.0f);  
    }  
}
```

10.3 力と加速度

$$\mathbf{f}_i^p = -\sum_{j \neq i} m (p_i + p_j) / (2\rho_j) \cdot \nabla W_{\text{spiky}}(r_{ij})$$

$$\mathbf{f}_i^v = \mu \sum_{j \neq i} m (\mathbf{v}_j - \mathbf{v}_i) / \rho_j \cdot \nabla^2 W_{\text{visc}}(r_{ij})$$

$$\mathbf{a}_i = (\mathbf{f}_i^p + \mathbf{f}_i^v) / \rho_i + \mathbf{g}$$

圧力力の $(p_i + p_j)/2$ は「2 粒子の圧力の平均」で、作用・反作用が対称になる（粒子 i が j を押す力と j が i を押す力が等しい）ようにした形です。 ∇W は j から i へ向かうので、圧力力は常に離れる向き（斥力）になります。

```

//-----
// ComputeForces
// 圧力力:  $f_p = \sum_j m (p_i + p_j) / (2 \rho_j) * 45 / (\pi h^6) (h - r)^2 * \hat{r}_{ij}$ 
//          ( $\hat{r}_{ij} = (x_i - x_j) / r$ 。粒子 j から離れる向きに押される)
// 粘性力:  $f_v = \mu \sum_j m (v_j - v_i) / \rho_j * 45 / (\pi h^6) (h - r)$ 
// 加速度:  $a_i = (f_p + f_v) / \rho_i + g$ 
//-----
void SPH::ComputeForces()
{
    const int n = m_count;
    const float mass = m_mass;
    const float h = m_h;
    const float spikyCoef = m_spikyGradCoef;
    const float viscCoef = m_viscLapCoef;
    const float mu = m_params.viscosity;
    const float g = m_params.gravity;

    const int maxN = m_params.maxNeighbors;

#pragma omp parallel for schedule(dynamic, 64)
    for (int i = 0; i < n; ++i)
    {
        const float pi = m_pressure[i];
        const XMFLOAT3 xi = m_pos[i];
        const XMFLOAT3 vi = m_vel[i];
        const int* list = &m_neighbors[static_cast<size_t>(i) * maxN];
        const int count = m_neighborCount[i];
        float fx = 0.0f, fy = 0.0f, fz = 0.0f;

        for (int k2 = 0; k2 < count; ++k2)
        {
            const int j = list[k2];
            const XMFLOAT3& xj = m_pos[j];
            const float rx = xi.x - xj.x, ry = xi.y - xj.y, rz = xi.z - xj.z;
            const float r2 = rx * rx + ry * ry + rz * rz;
            if (j == i || r2 < 1e-12f)
                continue; // 自分自身、または完全に重なった粒子は除外 (0 除算防止)

            const float r = std::sqrt(r2);
            const float invR = 1.0f / r;
            const float hr = h - r;
            const float rhoJ = m_density[j];
            const float invRhoJ = 1.0f / rhoJ;

            // 圧力力 (斥力)
            const float pTerm = mass * (pi + m_pressure[j]) * 0.5f * invRhoJ * spikyCoef * hr * hr *
invR;

            fx += pTerm * rx;
            fy += pTerm * ry;
            fz += pTerm * rz;

            // 粘性力 (周囲との速度差を減らす)
            const float vTerm = mu * mass * invRhoJ * viscCoef * hr;
            const XMFLOAT3& vj = m_vel[j];
            fx += vTerm * (vj.x - vi.x);
            fy += vTerm * (vj.y - vi.y);
            fz += vTerm * (vj.z - vi.z);
        }
    }
}

```

```

const float invRhoI = 1.0f / m_density[i];
float ax = fx * invRhoI, ay = fy * invRhoI, az = fz * invRhoI;

// 圧力・粘性による加速度の大きさを上限で頭打ちにする
// (壁際で密度が急上昇したときに粒子が弾き飛ばされるのを防ぐ。重力は別に足す)
const float amax = m_params.maxAcceleration;
const float a2 = ax * ax + ay * ay + az * az;
if (a2 > amax * amax)
{
    const float scale = amax / std::sqrt(a2);
    ax *= scale; ay *= scale; az *= scale;
}
m_acc[i] = XMFLOAT3(ax, ay - g, az);
}
}

```

実装では `spikyCoef * hr * hr * invR` に $(x_i - x_j)$ を掛けることで「大きさ × 単位ベクトル」を一度に計算しています。加速度の上限 (`maxAcceleration`) については第 12 章で説明します。

10.4 時間積分

加速度から速度と位置を進めます。もっとも簡単な**シンプレクティック・オイラー法**（速度を先に更新し、その新しい速度で位置を更新する）を使います。

$$\mathbf{v} \leftarrow \mathbf{v} + \mathbf{a}\Delta t, \quad \mathbf{x} \leftarrow \mathbf{x} + \mathbf{v}\Delta t$$

「位置を先に更新する」普通のオイラー法と比べて、エネルギーが増え続ける（爆発する）傾向が小さく、粒子系では標準的な選択です。

src/SPH.cpp

```

//-----
// Integrate
//-----
void SPH::Integrate()
{
    const int n = m_count;
    const float dt = m_params.timeStep;
    const float vmax = m_params.maxSpeed;

#pragma omp parallel for
    for (int i = 0; i < n; ++i)
    {
        XMFLOAT3& v = m_vel[i];
        XMFLOAT3& p = m_pos[i];
        const XMFLOAT3& a = m_acc[i];

        // 速度更新
        v.x += a.x * dt;
        v.y += a.y * dt;
        v.z += a.z * dt;

        // 速度の上限 (発散の安全弁)
        const float speed2 = v.x * v.x + v.y * v.y + v.z * v.z;
        if (speed2 > vmax * vmax)
        {

```

```

        const float scale = vmax / std::sqrt(speed2);
        v.x *= scale; v.y *= scale; v.z *= scale;
    }

    // 位置更新
    p.x += v.x * dt;
    p.y += v.y * dt;
    p.z += v.z * dt;

    ResolveCollision(i);
}
}

```

10.5 時間刻みと CFL 条件

Δt は大きいほど速く計算できますが、大きすぎると発散します。目安となるのが **CFL 条件**「1 ステップで情報が伝わる距離が h を超えてはならない」です。圧力波は音速 $c = \sqrt{k}$ で伝わるので、

$$\Delta t < 0.4 h / (c + v_{\max}) \approx 0.4 \times 0.6 / (45 + 10) \approx 0.0044 \text{ s}$$

本プログラムは $\Delta t = 0.003 \text{ s}$ です。1 フレーム（1/60 s）を複数のサブステップに分けて進めますが、処理落ちでフレーム時間が伸びるとサブステップが増えてさらに遅くなる悪循環（死のスパイラル）に陥ります。そこでサブステップ数に上限を設け、上限に達したら実時間より遅く進めます。

src/SPH.cpp

```

//-----
// Update
//   処理落ちで frameTime が大きくなったときにサブステップ数が増え続けて
//   さらに遅くなる（スパイラル）のを防ぐため、maxSubsteps で頭打ちにする。
//   その場合シミュレーションは実時間より遅く進む。
//-----
void SPH::Update(float frameTime)
{
    // 実時間に追いつくのに必要なサブステップ数を求め、上限で頭打ちにする
    int substeps = static_cast<int>(std::ceil(frameTime / m_params.timeStep));
    substeps = std::clamp(substeps, 1, m_params.maxSubsteps);
    for (int s = 0; s < substeps; ++s)
        Step();
}

```

10.6 パラメータの決め方

src/SPH.h

```

//-----
// SPHParams : シミュレーションの調整パラメータ
//-----
struct SPHParams
{
    // ---- 粒子の物理量 ----
    float particleSpacing = 0.30f; // 粒子の初期間隔 [m] (粒子径とみなす)
    float smoothingRadius = 0.60f; // 平滑化半径 h [m] (通常は間隔の 2 倍)
    float restDensity      = 1000.0f; // 静止密度 ρ0 [kg/m^3] (水)
}

```

```

float stiffness      = 2000.0f; // 圧力の硬さ k (大きいほど非圧縮に近づくが不安定になる)
float viscosity      = 0.40f;  // 粘性係数  $\mu$  (数値安定化のため実際の水より大きめ。流れをまとらせる)
float gravity        = 9.8f;    // 重力加速度 [m/s^2]
float maxSpeed       = 10.0f;   // 速度の上限 [m/s] (発散防止の安全弁)
float maxAcceleration = 250.0f; // 加速度の上限 [m/s^2] (壁際の圧力スパイクによる「蹴り」を抑える)

// ---- 時間刻み ----
float timeStep       = 0.003f; // 1 サブステップの時間 [s] (CFL 条件  $dt < 0.4 h / c$  を満たすこと)
int    maxSubsteps    = 5;      // 1 フレームで実行するサブステップ数の上限
                                   // (処理落ち時はサブステップを増やさず、実時間より遅く進める)

// ---- 境界との衝突 ----
float restitution    = 0.0f;    // 反発係数 ( $\theta =$  跳ね返らない)
float friction        = 0.010f; // 接地時に 1 サブステップあたり速度に掛ける減衰率 (地面との摩擦)
float contactZone     = 0.30f;  // 地面からこの高さ [m] 以内を「接触ゾーン」とし、
                                   // 地面に近いほど強く減衰させて跳ね上がりを抑える
float contactDamping  = 0.06f;  // 接触ゾーンの最下部で 1 サブステップあたりに掛ける減衰率
float airDrag         = 0.015f; // 孤立粒子 (飛沫) に 1 サブステップあたり掛ける空気抵抗
int    sparseNeighbors = 8;     // 近傍がこの数未満なら孤立粒子とみなす (自分自身を含む)

// ---- エミッタ (粒子の投入口) ----
int    maxParticles   = 1200;   // 最大粒子数
int    maxNeighbors   = 96;     // 1 粒子あたり記録する近傍粒子数の上限
int    emitWidth      = 14;     // 1 回の投入で横に並べる粒子数
int    emitHeight     = 4;      // 1 回の投入で縦に積む粒子数
float  emitSpeed       = 4.0f;  // 投入時の初速 [m/s] (下流 +z 方向)
float  emitZ           = -Terrain::HALF_SIZE + 2.5f; // 投入位置の z (上流の端付近)
float  emitLift        = 0.8f;  // 地面から投入位置までの高さ [m]
};

```

量	値	決め方
粒子間隔 s	0.30 m	谷底幅 (≈ 3 m) に約 10 個並ぶ粒度。細かくすると粒子数は 3 乗で増える
平滑化半径 h	$2s = 0.60$ m	静止状態で約 33 個の近傍。少なすぎると密度がガタつき、多いと重い
質量 m	$\rho_0 s^3 = 27$ kg	粒子 1 個の占める体積 $\times$ 水の密度。これで静止時に $\rho \approx \rho_0$ になる
剛性 k	2000	深さ H の水柱の圧縮率は $\rho g H / k \approx 0.5\% / \text{m}$ 。大きくすると Δt を小さくする必要
粘性 μ	0.40	実際の水より大きいが、粒子のばらつきを抑え流れをまとめる。安定条件の目安は < 1

演習 10

1. `stiffness` を 500 にすると水が「柔らかく」なって沈み込みます。5000 にすると何が起きますか (Δt との関係を考えてください)。
2. 圧力の `std::max(rho - rho0, 0)` を外して負圧を許し、粒子が塊になる様子を観察してください。
3. `timeStep` を 0.006 にして CFL 条件を破ると、どのように発散するか観察してください。

近傍探索 — 空間ハッシュとカウンティングソート

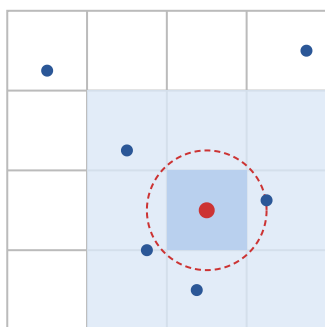
SPH の計算時間のほとんどは「半径 h 内の粒子を見つける」ことに費やされます。この章では、全対全の $O(N^2)$ を避けるための格子分割、無限空間を固定サイズの表に写す空間ハッシュ、動的メモリ確保なしで格子を作るカウンティングソートを学びます。

11.1 なぜ工夫が必要か

粒子 12,000 個で全対全の距離を調べると 1.4 億回/ステップ、1 秒間に 300 ステップなら 430 億回で、CPU では到底間に合いません。しかし距離 h 以上の粒子は寄与 0 なので、「近くにいる可能性のある粒子」だけを調べれば十分です。

11.2 一様格子と 27 セル走査

空間を一辺 h の立方体セルに切り、各粒子をセルに登録します。粒子 i の近傍は、 i のいるセルと隣接する 26 セル、合計 27 セルの中にしかありません。



セルの一辺 = h
 粒子 (赤) の探索半径 h の円は
 周囲 3×3 (3D では $3 \times 3 \times 3$) セルに収まる
 候補: $27 \text{ セル} \times \text{平均 } 8 \text{ 個} \approx 216 \text{ 個}$
 → 距離判定で約 33 個に絞る
 全対全 12,000 個 → 216 個 (55 倍速)

図 11-1 一様格子による近傍候補の絞り込み (2D で図示)

11.3 空間ハッシュ

地形全体を格子で埋めると 100 万セル以上になり、毎ステップのクリアが重くなります。代わりにセル座標をハッシュ関数で固定サイズの表 (粒子数の 2 倍程度) に写します。空のセルはメモリを消費しません。

$$(c_x, c_y, c_z) = \lfloor \mathbf{x} / h \rfloor, \text{ key} = (c_x \cdot 73856093 \oplus c_y \cdot 19349663 \oplus c_z \cdot 83492791) \& (\text{tableSize} - 1)$$

3 つの大きな素数を掛けて XOR する Teshner らのハッシュです。表のサイズを 2 のべき乗にすると、剰余 ($\%$ 、遅い) をビットマスク ($\&$ 、速い) で置き換えられます。

src/SPH.cpp

```
//-----
// HashCell
//-----
```

```
uint32_t SPH::HashCell(int cx, int cy, int cz) const
{
    const uint32_t h = (static_cast<uint32_t>(cx) * 73856093u)
        ^ (static_cast<uint32_t>(cy) * 19349663u)
        ^ (static_cast<uint32_t>(cz) * 83492791u);
    return h & m_tableMask;
}
```

11.4 カウンティングソート

「同じハッシュ値の粒子をまとめて取り出したい」を、リストや動的配列なしで実現するのがカウンティングソートです。

① 粒子ごとの key

2	0	2	1	0	2
---	---	---	---	---	---

粒子 $i = 0 \dots 5$

④ 粒子番号を開始位置から順に書き込む

1	4	3	0	2	5
---	---	---	---	---	---

sorted[]: key0 の粒子 | key1 | key2 の粒子

② key ごとに数える → ③ 累積和 = 開始位置

count: [2, 1, 3]

cellStart: [0, 2, 3, 6]

key0 は 0~1, key1 は 2, key2 は 3~5 に入る

key = k の粒子は sorted[cellStart[k] .. cellStart[k+1]] で連続

毎ステップ $O(N + \text{tableSize})$ で再構築、メモリ確保なし

図 11-2 カウンティングソートによる格子構築

src/SPH.cpp

```
//-----
// BuildGrid
// アルゴリズム (カウンティングソート):
//   1. 各粒子のセル座標とハッシュ値を計算 (並列)
//   2. ハッシュ値ごとの粒子数を数える count[key]++
//   3. 累積和を取り、各ハッシュ値の開始位置にする cellStart[key] = Σ count[0..key-1]
//   4. 粒子番号を開始位置から順に書き込む sorted[cellStart[key] + fill[key]++] = i
//-----
void SPH::BuildGrid()
{
    const int n = m_count;

    // 1. セル座標とハッシュ値
#pragma omp parallel for
    for (int i = 0; i < n; ++i)
    {
        m_cell[i] = CellCoord(m_pos[i]);
        m_cellKey[i] = HashCell(m_cell[i].x, m_cell[i].y, m_cell[i].z);
    }

    // 2. ハッシュ値ごとの粒子数
    std::fill(m_cellStart.begin(), m_cellStart.end(), 0u);
    for (int i = 0; i < n; ++i)
        ++m_cellStart[m_cellKey[i] + 1]; // +1 にしておくで次の累積和でそのまま開始位置になる

    // 3. 累積和 → 開始位置
    for (uint32_t k = 0; k < m_tableSize; ++k)
        m_cellStart[k + 1] += m_cellStart[k];

    // 4. 振り分け
    std::fill(m_cellFill.begin(), m_cellFill.end(), 0u);
    for (int i = 0; i < n; ++i)
```

```

{
    const uint32_t key = m_cellKey[i];
    m_sorted[m_cellStart[key] + m_cellFill[key]++] = i;
}
}

```

11.5 近傍の走査とハッシュ衝突

27 セルそれぞれのハッシュ値を計算し、`sorted` の該当区間を走査します。ここで 1 つ罣があります。異なるセルが同じハッシュ値を持つ（衝突する）と、27 セルのうち 2 つが同じ区間を指し、同じ粒子を 2 回数えてしまいます。候補粒子の実際のセル座標が「今調べているセル」と一致するか確認して除外します。

src/SPH.cpp

```

//-----
// ForEachNeighbor
//-----
template <class F>
void SPH::ForEachNeighbor(int i, F&& func) const
{
    const XMFLOAT3 pi = m_pos[i];
    const XMINT3    ci = m_cell[i];

    for (int dz = -1; dz <= 1; ++dz)
    {
        for (int dy = -1; dy <= 1; ++dy)
        {
            for (int dx = -1; dx <= 1; ++dx)
            {
                const int cx = ci.x + dx;
                const int cy = ci.y + dy;
                const int cz = ci.z + dz;
                const uint32_t key = HashCell(cx, cy, cz);

                const uint32_t begin = m_cellStart[key];
                const uint32_t end   = m_cellStart[key + 1];
                for (uint32_t k = begin; k < end; ++k)
                {
                    const int j = m_sorted[k];

                    // ハッシュ衝突対策：本当に同じセルの粒子かを確認
                    const XMINT3& cj = m_cell[j];
                    if (cj.x != cx || cj.y != cy || cj.z != cz)
                        continue;

                    const float rx = pi.x - m_pos[j].x;
                    const float ry = pi.y - m_pos[j].y;
                    const float rz = pi.z - m_pos[j].z;
                    const float r2 = rx * rx + ry * ry + rz * rz;
                    if (r2 < m_h2)
                        func(j, rx, ry, rz, r2);
                }
            }
        }
    }
}

```

11.6 近傍リストのキャッシュ

密度計算と力計算は同じ近傍を使います。走査を 1 回にまとめ、粒子ごとに近傍番号を最大 96 個記録しておく、以降の計算は単純な配列の走査になります。この変更で実測約 3 倍高速化しました。

src/SPH.cpp

```
//-----  
// BuildNeighborLists  
// 近傍数が maxNeighbors を超えた分は捨てる（極端に圧縮された場合のみ起こる）。  
//-----  
void SPH::BuildNeighborLists()  
{  
    const int n = m_count;  
    const int maxN = m_params.maxNeighbors;  
  
#pragma omp parallel for schedule(dynamic, 64)  
    for (int i = 0; i < n; ++i)  
    {  
        int* list = &m_neighbors[static_cast<size_t>(i) * maxN];  
        int count = 0;  
        ForEachNeighbor(i, [&](int j, float, float, float, float)  
        {  
            if (count < maxN)  
                list[count++] = j;  
        });  
        m_neighborCount[i] = count;  
    }  
}
```

ポイント 性能改善の順序は「アルゴリズム ($O(N^2) \rightarrow O(N)$) $\rightarrow$ データ構造（メモリ確保なし、連続アクセス） $\rightarrow$ 細かい最適化（剰余をマスクに、平方根を避ける）」です。順序を逆にすると効果が薄くなります。

演習 11

1. `ForEachNeighbor` のセル座標一致チェックを外し、ハッシュ表サイズを小さく（例えば 1024）して、密度が過大になる様子を観察してください。
2. `smoothingRadius` を 0.9 にすると近傍数と処理時間はどう変わりますか。理論値（体積比）と比べてください。
3. 27 セルではなく「自分のセルだけ」を走査するように変えると、何が起きますか。

境界処理・安定化・エミッタ

流体は地形や壁に当たります。この章では地形との衝突処理、粒子が谷から飛び出さないようにするための 3 つの安定化手法、そして水を投入し続けるエミッタと排水の仕組みを学びます。

12.1 地形との衝突

粒子の中心が「地面の高さ + 粒子半径」より低くなったら、その高さまで押し戻し、地面にめり込む向き速度成分を取り除きます。

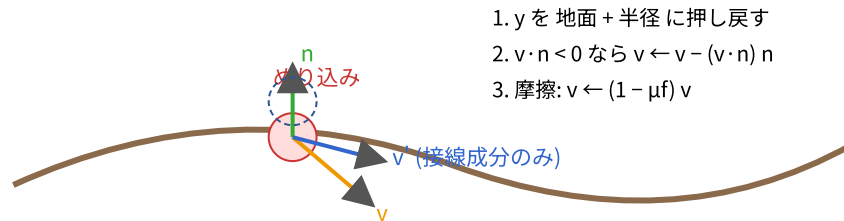


図 12-1 地形との衝突処理

$v \leftarrow v - (1 + e)(v \cdot n)n$ ($v \cdot n < 0$ のとき。 e = 反発係数、本プログラムは 0)

$(v \cdot n)n$ は速度の法線方向成分です。これを引くと、地面に沿って滑る成分だけが残ります。 $e = 1$ なら完全弾性衝突（跳ね返る）、0 なら跳ね返らずに滑ります。

12.2 飛び出しを防ぐ 3 つの安定化

基本の衝突処理だけでは、壁際で密度が急上昇したときの圧力の「蹴り」で粒子が谷の外へ飛散しました。次の 3 手法で解消しています。

手法	内容	パラメータ
加速度クランプ	圧力・粘性による加速度の大きさに上限。1 ステップの速度変化を 0.75 m/s に制限（重力は別に加算）	<code>maxAcceleration = 250</code>
接触ゾーン減衰	地面から 0.3 m 以内では、地面に近いほど強く速度を減衰（最下部で 6%/ステップ、上端で 0）。跳ね上がりを吸収	<code>contactZone</code> , <code>contactDamping</code>
孤立粒子の空気抵抗	近傍が 8 個未満の粒子（飛沫）に 1.5%/ステップの減衰。勢いを失ってすぐ流れに戻る	<code>airDrag</code> , <code>sparseNeighbors</code>

src/SPH.cpp

```
//-----
// ResolveCollision
```

```

// アルゴリズム:
// 1. 側壁 (x = ±limit) と上流の壁 (z = -limit) を越えたら壁面に戻し、
//    壁に向かう速度成分を 0 にする
// 2. 地形: 粒子の中心が「地面の高さ + 粒子半径」より低ければ
//    その高さまで持ち上げ、地面法線 n に対して
//     $v \leftarrow v - (1 + e)(v \cdot n) n$  ( $v \cdot n < 0$  のときのみ)
//    で地面にめり込む速度成分を除去する (e = 反発係数)。
//    さらに接地中はわずかな摩擦として速度全体を (1 - friction) 倍する。
// 3. 接触ゾーン: 地面から contactZone 以内にある粒子は、地面に近いほど強く
//    減衰させる (最下部で contactDamping)。これにより壁に当たった粒子が
//    跳ね上がって谷の外へ飛び出すのを抑える。
// 4. 空気抵抗: 近傍粒子が sparseNeighbors 未満の孤立粒子 (飛沫) には
//    airDrag の減衰を掛け、勢いを失ってすぐ流れに戻るようにする。
//-----
void SPH::ResolveCollision(int i)
{
    XMFLOAT3& p = m_pos[i];
    XMFLOAT3& v = m_vel[i];
    const float radius = m_params.particleSpacing * 0.5f;
    const float limit = Terrain::HALF_SIZE - 0.5f;

    // 1. 側壁・上流の壁
    if (p.x < -limit) { p.x = -limit; if (v.x < 0.0f) v.x = 0.0f; }
    if (p.x > limit) { p.x = limit; if (v.x > 0.0f) v.x = 0.0f; }
    if (p.z < -limit) { p.z = -limit; if (v.z < 0.0f) v.z = 0.0f; }

    // 2. 地形
    const float floorY = m_terrain->GetHeight(p.x, p.z) + radius;
    if (p.y < floorY)
    {
        p.y = floorY;
        const XMFLOAT3 n = m_terrain->GetNormal(p.x, p.z);
        const float vn = v.x * n.x + v.y * n.y + v.z * n.z;
        if (vn < 0.0f)
        {
            const float s = (1.0f + m_params.restitution) * vn;
            v.x -= s * n.x;
            v.y -= s * n.y;
            v.z -= s * n.z;
        }
        const float damp = 1.0f - m_params.friction;
        v.x *= damp; v.y *= damp; v.z *= damp;
    }

    // 3. 接触ゾーン: 地面からの高さ (0 ~ contactZone) に応じて減衰を強める
    //    t = 1 (地面に接している) → contactDamping、t = 0 (ゾーン上端) → 0
    const float height = p.y - floorY;
    if (height < m_params.contactZone)
    {
        const float t = 1.0f - std::max(height, 0.0f) / m_params.contactZone;
        const float damp = 1.0f - m_params.contactDamping * t;
        v.x *= damp; v.y *= damp; v.z *= damp;
    }

    // 4. 空気抵抗: 孤立した粒子 (飛沫) の勢いを削ぐ
    if (m_neighborCount[i] < m_params.sparseNeighbors)
    {
        const float damp = 1.0f - m_params.airDrag;
        v.x *= damp; v.y *= damp; v.z *= damp;
    }
}

```

```
}  
}
```

コラム: 境界密度の欠損 地面の下には粒子がないので、接地している粒子の密度は本来より低く見積もられ、圧力が弱くなって沈み込みやすくなります。厳密には地形表面に「境界粒子」を並べて密度に寄与させる方法（Akinci ら 2012）がありますが、本プログラムは衝突処理と接触ゾーン減衰でこの効果を吸収しています。

12.3 エミッタと排水

上流の谷底に、粒子の「板」（横 14 × 縦 4）を一定間隔で置き、初速を与えて流します。板同士が重ならないよう、投入間隔は「板が 1 粒子間隔ぶん進む時間 = s/v_{emit} 」にします。完全な格子配置は不自然な規則性を生むので、小さなランダムなずれ（ジッタ）を加えます。

src/SPH.cpp

```
//-----  
// EmitSheet  
// 谷の中心 x を中心に emitWidth 個を横並びにし、それを emitHeight 段積む。  
// 各粒子には小さなランダムなずれ（ジッタ）を加えて、  
// 完全な格子配置による不自然な規則性を崩す。  
//-----  
void SPH::EmitSheet()  
{  
    const float s = m_params.particleSpacing;  
    const float z = m_params.emitZ;  
    const float cx = m_terrain->GetValleyCenterX(z);  
  
    for (int iy = 0; iy < m_params.emitHeight; ++iy)  
    {  
        for (int ix = 0; ix < m_params.emitWidth; ++ix)  
        {  
            if (m_count >= m_params.maxParticles)  
                return;  
  
            const float jitterX = (XorShift32(m_rng) - 0.5f) * s * 0.3f;  
            const float jitterY = (XorShift32(m_rng) - 0.5f) * s * 0.3f;  
  
            const float x = cx + (ix - (m_params.emitWidth - 1) * 0.5f) * s + jitterX;  
            const float ground = m_terrain->GetHeight(x, z);  
            const float y = ground + m_params.emitLift + iy * s + jitterY;  
  
            const int i = m_count++;  
            m_pos[i] = XMFLOAT3(x, y, z);  
            m_vel[i] = XMFLOAT3(0.0f, 0.0f, m_params.emitSpeed);  
            m_acc[i] = XMFLOAT3(0.0f, 0.0f, 0.0f);  
            m_density[i] = m_params.restDensity;  
            m_pressure[i] = 0.0f;  
        }  
    }  
}
```

下流端に達した粒子は削除します。配列の途中を消すと詰め直しに $O(N)$ かかるので、「末尾の粒子と入れ替えて末尾を切る」 $O(1)$ の方法を使います（順序は保たれませんが問題ありません）。

```
//-----
// RemoveDrained
// 有効粒子は配列の先頭に詰めて管理しているので、
// 消す粒子を末尾の粒子で上書きし m_count を 1 減らす (順序は保たなくてよい)。
//-----
void SPH::RemoveDrained()
{
    const float drainZ = Terrain::HALF_SIZE - 0.5f;
    int i = 0;
    while (i < m_count)
    {
        const XMFLOAT3& p = m_pos[i];
        const bool drained = (p.z > drainZ) || (p.y < -60.0f);
        if (drained)
        {
            const int last = m_count - 1;
            m_pos[i] = m_pos[last];
            m_vel[i] = m_vel[last];
            m_acc[i] = m_acc[last];
            m_density[i] = m_density[last];
            m_pressure[i] = m_pressure[last];
            --m_count;
            // i は進めない (入れ替えた粒子を再チェック)
        }
        else
        {
            ++i;
        }
    }
}
```

演習 12

1. `restitution` を 0.5 にして、粒子が地面で跳ねる様子を観察してください。
2. `contactDamping` を 0 にすると飛び出しが再発しますか。0.03 ではどうでしょう。
3. エミッタの `emitSpeed` を 8 にすると投入間隔が短くなります。1 秒あたりの投入粒子数を計算し、最大粒子数に達するまでの時間を予測してから確かめてください。

OpenMP による並列化

SPH の粒子ごとの計算は互いに独立なので、CPU の複数コアで同時に処理できます。この章では、コンパイラ指示子だけで並列化できる OpenMP の使い方と、並列化してよい処理・してはいけない処理の見分け方を学びます。

13.1 OpenMP とは

OpenMP は、`#pragma omp parallel for` を `for` 文の前に書くだけで、その繰り返しを複数スレッドに分配してくれる仕組みです。Visual Studio では `/openmp` オプション（プロジェクト設定「OpenMP のサポート」）を有効にします。

```
#pragma omp parallel for schedule(dynamic, 64)
for (int i = 0; i < n; ++i)
{
    // i 番目の粒子の処理。他の i の結果に依存しないこと
}
```

- **schedule(dynamic, 64):** 64 個ずつの塊をスレッドが空き次第取っていく。粒子ごとに近傍数が違い処理時間が不均一なので、静的な等分割より効率が良い。
- ループ変数は `int` の符号付き整数、ループ回数はループ開始前に決まっている必要がある。

13.2 データ競合 — 並列化してよい処理の条件

2 つのスレッドが同じ変数に同時に書き込むと、結果が壊れます（データ競合）。並列化してよいのは「各 i が自分の要素にだけ書き、他の要素は読むだけ」の処理です。

処理	並列	理由
セル座標・ハッシュ計算	○	<code>m_cell[i]</code> , <code>m_cellKey[i]</code> に書くだけ
カウント・累積和・振り分け	×	<code>cellStart[key]++</code> は複数の i が同じ <code>key</code> に書く（競合）。粒子数が少ないので逐次で十分
近傍リスト作成	○	自分のリストに書くだけ。位置は読むだけ
密度・圧力	○	<code>m_density[i]</code> , <code>m_pressure[i]</code> に書くだけ
力	○	<code>m_acc[i]</code> に書くだけ。他粒子の圧力・速度は読むだけ
積分・衝突	○	<code>m_pos[i]</code> , <code>m_vel[i]</code> 。地形の高さマップは読むだけ
排水（削除）	×	配列の詰め直しは順序に依存する

「対称な力を両方の粒子に加算する」最適化はできない 作用・反作用を利用して i - j の対を 1 回だけ計算し両方に足す方法は、`m_acc[j]` への書き込みが競合するため、そのままでは並列化できません。本プログラムは各粒子が自分の力だけを計算する（対を 2 回計算する）代わりに、競合ゼロで並列化しています。

13.3 効果

8 論理コアの CPU で、12,000 粒子・5 サブステップ/フレームの SPH が 1 フレーム 8～15 ms で計算できています。粒子数に対して処理時間はほぼ線形に増えるので、コア数が多いマシンではそのぶん粒子を増やせます。さらに 10 倍以上を目指すなら GPU のコンピュータシエダーへの移植になります（第 18 章）。

演習 13

1. プロジェクト設定で OpenMP を無効にし（または `#pragma` をコメントアウト）、FPS の変化を測ってください。
2. `schedule(static)` に変えると速くなりますか、遅くなりますか。理由を考えてください。
3. 環境変数 `OMP_NUM_THREADS=2` で起動して、スレッド数と性能の関係を調べてください。

第 IV 部

表現編

計算した粒子を、どう「見せる」か。

球としての描画、水面の生成、そしてカメラと操作。

粒子の描画 — インスタンスングとスフィア・インポスター

12,000 個の球をどう効率よく描くか。この章では、1 回の描画命令で大量の同じ形を描くインスタンスングと、平らな四角形をピクセルシェーダーで「本物の球」に見せるスフィア・インポスターの技法を学びます。

14.1 課題 — 球を 12,000 個描く

球を三角形メッシュで作ると、粗くても 1 個 100 三角形ほど必要で、12,000 個で 120 万三角形になります。しかも遠くの粒子は数ピクセルにしか映らず、ほとんどが無駄になります。そこで「カメラに正対する四角形を描き、ピクセルシェーダーで球の形に削る」スフィア・インポスター (impostor = 替え玉) を使います。1 粒子 2 三角形で済み、どんな距離でも輪郭が滑らかです。

14.2 ビルボードの生成

粒子データ (位置と速さ) は `StructuredBuffer<float4>` として GPU に置き、`DrawInstanced(4, N)` で「4 頂点の四角形を N 個」描きます。頂点シェーダーは `SV_InstanceID` で粒子を、`SV_VertexID` で四隅を選びます。四角形はビュー空間で作るので、常にカメラに正対します (ビルボード)。

shaders/Particle.hlsl

```
//-----
// 関数名 : VSMain
// 概要   : 粒子 iid の中心をビュー空間に変換し、頂点 vid に対応する四角形の角を出力する
// 引数   : vid : SV_VertexID (0~3、トライアングルストリップの順)
//          iid : SV_InstanceID (粒子番号)
// 戻り値 : PSIn
//-----
PSIn VSMain(uint vid : SV_VertexID, uint iid : SV_InstanceID)
{
    float4 p = Particles[iid];
    float3 centerV = mul(float4(p.xyz, 1.0), View).xyz;

    // トライアングルストリップ用の 4 隅: 左下, 左上, 右下, 右上
    float2 corners[4] = { float2(-1, -1), float2(-1, 1), float2(1, -1), float2(1, 1) };
    float2 c = corners[vid];

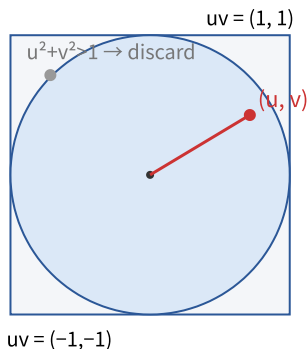
    float r = Params.x; // 粒子の描画半径
    float3 posV = centerV + float3(c * r, 0.0); // 常にカメラに正対する四角形

    PSIn o;
    o.posH = mul(float4(posV, 1.0), Proj);
    o.uv    = c;
    o.centerV = centerV;
    o.speed  = p.w;
    return o;
}
```

src/FluidRenderer.cpp

```
//-----  
// DrawSpheres  
// 頂点バッファなし: IASetVertexBuffers を呼ばず、入力レイアウトも nullptr。  
// DrawInstanced(4, count): 4 頂点のトライアングルストリップを粒子数だけ描く。  
//-----  
void FluidRenderer::DrawSpheres(Graphics& gfx, UINT count, ID3D11PixelShader* ps)  
{  
    ID3D11DeviceContext* ctx = gfx.GetContext();  
  
    ctx->IASetInputLayout(nullptr);  
    ctx->IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_TRIANGLESTRIP);  
    ctx->VSSetShader(m_vsParticle.Get(), nullptr, 0);  
    ctx->PSSetShader(ps, nullptr, 0);  
  
    ID3D11ShaderResourceView* srv = m_particleSRV.Get();  
    ctx->VSSetShaderResources(0, 1, &srv);  
  
    ctx->RSSetState(gfx.GetRasterizerNoCull());  
    ctx->OMSetDepthStencilState(gfx.GetDepthStateDefault(), 0);  
    ctx->OMSetBlendState(gfx.GetBlendOpaque(), nullptr, 0xFFFFFFFF);  
  
    ctx->DrawInstanced(4, count, 0, 0);  
  
    // SRV を外しておく (同じリソースを後で RTV にバインドするときの競合防止)  
    ID3D11ShaderResourceView* nullSrv = nullptr;  
    ctx->VSSetShaderResources(0, 1, &nullSrv);  
}
```

14.3 球面の再構成と深度の書き込み



ピクセルシェーダーでの処理

$$r^2 = u^2 + v^2$$

$r^2 > 1$ なら球の外 → 描かない (discard)

$$\text{法線 } n = (u, v, -\sqrt{1 - r^2})$$

(左手系: カメラ側の半球は $z < 0$)

球面上の点 $p = \text{中心} + n \cdot \text{半径}$

p を射影して深度を `SV_Depth` に書く

→ 球同士・地形と正しく交差する

図 14-1 四角形上の座標 (u, v) から球面を復元する

四角形の中の座標 (u, v) ($-1 \sim 1$) を「球を正面から見た図」とみなすと、球面上の高さは $\sqrt{1 - u^2 - v^2}$ です。これで法線と、球面上の 3D 位置が求まります。位置を射影すれば正しい深度値が得られ、`SV_Depth` に書き込むと、平らな四角形なのに球として深度テストされます。

shaders/Particle.hlsl

```
//-----  
// 関数名 : SphereSurface  
// 概要 : 四角形内の座標から球面上の点と法線を求める (共通処理)  
// 引数 : i : ピクセル入力  
// normalV : [出力] 球面の法線 (ビュー空間)  
// posV : [出力] 球面上の点 (ビュー空間)
```

```
//          depth   : [出力] 正規化デバイス座標の深度 (SV_Depth 用)
// 戻り値 : true = 球の内側, false = 球の外側 (discard すべき)
//-----
bool SphereSurface(PSPIn i, out float3 normalV, out float3 posV, out float depth)
{
    float r2 = dot(i.uv, i.uv);
    normalV = 0; posV = 0; depth = 0;
    if (r2 > 1.0)
        return false;

    // 左手系のビュー空間ではカメラは +z を向くので、カメラ側の半球は z < 0
    normalV = float3(i.uv, -sqrt(1.0 - r2));
    posV     = i.centerV + normalV * Params.x;

    float4 clip = mul(float4(posV, 1.0), Proj);
    depth = clip.z / clip.w;
    return true;
}
```

SV_Depth のコスト ピクセルシェーダーが深度を書くと、GPU は「シェーダー実行前に深度テストで捨てる」最適化（早期 Z）を使えなくなります。粒子 1 万個程度では問題になりませんが、画面を埋め尽くすような使い方では注意が必要です。

14.4 速度による着色

参考動画に合わせ、速さを「濃青 → 水色 → 緑 → 黄 → 赤」（Jet 風カラーマップ）に対応させます。5 つの制御色を等間隔に置いた区間線形補間です。

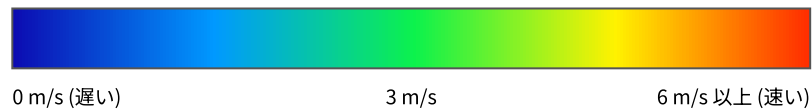


図 14-2 速度カラーマップ (JetColor)

shaders/Common.hlsl

```
//-----
// 関数名 : JetColor
// 概要   : 0~1 の値を「青 → 水色 → 緑 → 黄 → 赤」のグラデーションに変換する
//          (参考動画の速度による着色を再現)
// 引数   : t : 0~1 (0 = 遅い, 1 = 速い)
// 戻り値 : RGB (各 0~1)
//-----
float3 JetColor(float t)
{
    t = saturate(t);
    // 5 個の制御色を等間隔に置き、区間ごとに線形補間する
    const float3 c0 = float3(0.05, 0.05, 0.70); // 濃い青
    const float3 c1 = float3(0.00, 0.60, 1.00); // 水色
    const float3 c2 = float3(0.05, 0.95, 0.30); // 緑
    const float3 c3 = float3(1.00, 0.95, 0.00); // 黄
    const float3 c4 = float3(1.00, 0.15, 0.00); // 赤

    float s = t * 4.0;
    if (s < 1.0) return lerp(c0, c1, s);
    if (s < 2.0) return lerp(c1, c2, s - 1.0);
    if (s < 3.0) return lerp(c2, c3, s - 2.0);
    if (s < 4.0) return lerp(c3, c4, s - 3.0);
    return c4;
}
```

```

    if (s < 3.0) return lerp(c2, c3, s - 2.0);
    return lerp(c3, c4, s - 3.0);
}

```

SPH の速度は粒子ごとに細かく揺らぐため、そのまま着色すると色がまだらになります。CPU 側で「自分と近傍の速さの平均」を描画用の速さとして渡し、なめらかな色分布にしています（物理計算には影響しません）。

src/SPH.cpp

```

//-----
// GetRenderData
// アルゴリズム:
//   1. 各粒子の速さ |v_i| を計算
//   2. 直前のステップで作った近傍リストを使い、自分と近傍の速さの単純平均を取る
//      (近傍リストは RemoveDrained で粒子が入れ替わると一部ずれるが、
//       描画の色にしか使わないので許容する。範囲外の番号だけは除外する)
//-----
const std::vector<XMFLOAT4>& SPH::GetRenderData()
{
    const int n = m_count;
    const int maxN = m_params.maxNeighbors;
    m_renderData.resize(n);

    // 1. 速さ
#pragma omp parallel for
    for (int i = 0; i < n; ++i)
    {
        const XMFLOAT3& v = m_vel[i];
        m_speed[i] = std::sqrt(v.x * v.x + v.y * v.y + v.z * v.z);
    }

    // 2. 近傍平均
#pragma omp parallel for
    for (int i = 0; i < n; ++i)
    {
        const int* list = &m_neighbors[static_cast<size_t>(i) * maxN];
        const int count = m_neighborCount[i];
        float sum = m_speed[i];
        int num = 1;
        for (int k = 0; k < count; ++k)
        {
            const int j = list[k];
            if (j >= 0 && j < n)
            {
                sum += m_speed[j];
                ++num;
            }
        }
        const XMFLOAT3& p = m_pos[i];
        m_renderData[i] = XMFLOAT4(p.x, p.y, p.z, sum / num);
    }
    return m_renderData;
}

```

shaders/Particle.hlsl

```

//-----
// 関数名 : PSColor
// 概要   : 速度で着色した球を描く。拡散反射 + 鏡面反射でつやのある見た目にする。
// 引数   : i : ピクセル入力

```

```
// 戻り値 : 色と深度
//-----
PSColorOut PSColor(PSin i)
{
    float3 n, posV; float depth;
    if (!SphereSurface(i, n, posV, depth))
        discard;

    float3 L = normalize(LightDirV.xyz);
    float3 V = normalize(-posV);           // 表面からカメラへ
    float3 H = normalize(L + V);          // ハーフベクトル (Blinn-Phong)

    float diff = saturate(dot(n, L));
    float spec = pow(saturate(dot(n, H)), 40.0);

    float3 base = JetColor(i.speed / MAX_SPEED_FOR_COLOR);
    float3 col = base * (0.35 + 0.75 * diff) + spec * 0.5;

    PSColorOut o;
    o.color = float4(col, 1.0);
    o.depth = depth;
    return o;
}
```

演習 14

1. PSColor の discard をコメントアウトし、四角形がそのまま見えることを確認してください。
2. SphereSurface で o.depth を書かない (sv_Depth を使わない) ようにすると、球同士の重なりがどう変わりますか。
3. 色を速さではなく高さ y で決めるように変えてみましょう。

表面生成 — スクリーンスペース流体レンダリング

粒子の集まりを「1 枚の水面」として見せるには、粒子の間を滑らかにつなぐ面が必要です。この章では、3D のメッシュを作らずに画面空間だけで水面を作る手法（van der Laan ら 2009）を、深度パス・ぼかしパス・合成パスの順に学びます。

15.1 2 つのアプローチ

- **陽なメッシュ生成:** 粒子から密度場を作り、Marching Cubes で等値面のメッシュを抽出する。視点によらない正確な面が得られるが、毎フレーム 3D 格子を走査するため重い。
- **スクリーンスペース法:** 粒子を球として深度だけ描き、画面上でぼかして法線を復元する。画面のピクセル数にしか依存せず軽い。見える面しか作らないが、リアルタイム表示には十分。

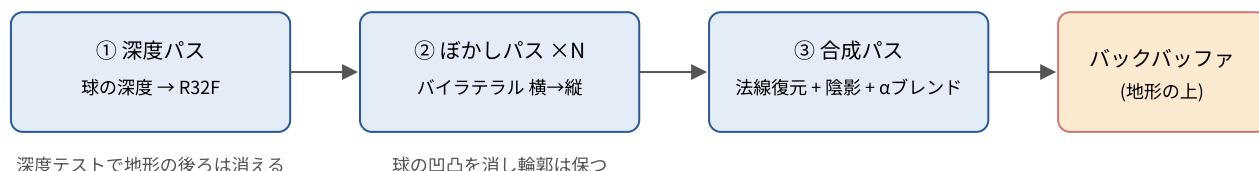


図 15-1 スクリーンスペース流体レンダリングの 3 パス

15.2 深度パス

第 14 章の球インポスターを使い、色の代わりに**ビュー空間の z** を `R32_FLOAT` テクスチャに書きます。メインの深度バッファ（地形描画済み）と組で描くので、地形の後ろの粒子は自動的に消えます。テクスチャは 0 でクリアし「0 = 流体なし」とします。表面用の球半径は粒子間隔の 1.3 倍にして、隣の球と重なり隙間を作らないようにします。

shaders/Particle.hlsl

```
//-----
// 関数名 : PSDepth
// 概要   : 表面生成用に球面のビュー空間  $z$  を書き出す。
//         深度テストにより地形の後ろや他の球の後ろは自動的に隠れる。
// 引数   : i : ピクセル入力
// 戻り値 : ビュー空間  $z$  と深度
//-----
PSDepthOut PSDepth(PSIn i)
{
    float3 n, posV; float depth;
    if (!SphereSurface(i, n, posV, depth))
        discard;

    PSDepthOut o;
    o.viewZ = posV.z;
    o.depth = depth;
    return o;
}
```

15.3 バイラテラルぼかし

深度テクスチャをそのままぼかすと、流体の輪郭で背景（深度 0）と混ざって縮み、また流体同士が重なる段差も溶けてしまいます。バイラテラルフィルタは「距離が近く、かつ値も近い」画素だけを平均するフィルタで、輪郭を保ったまま内部を滑らかにできます。

$$w_k = \exp(-k^2 / 2\sigma_s^2) \cdot \exp(-(d_k - d_0)^2 / 2\sigma_d^2), \quad d = \sum w_k d_k / \sum w_k$$

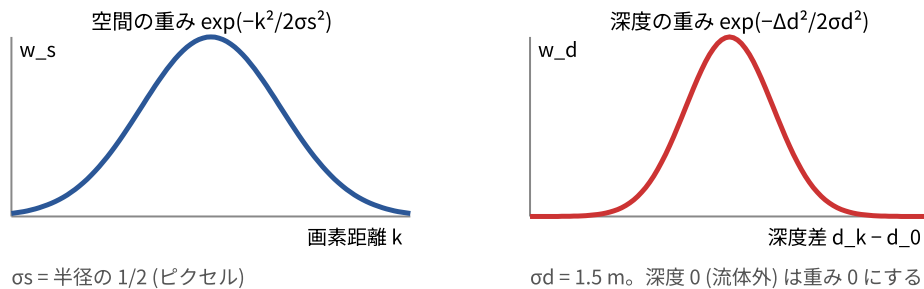


図 15-2 バイラテラルフィルタの 2 つの重み。両方の積が最終的な重みになる

実装上の工夫が 3 つあります。

- **透視補正:** ぼかし半径をワールド単位（0.9 m）で決め、それを画面に投影したピクセル数を使う。近い流体は広く、遠い流体は狭くぼかされ、距離によらず同じ滑らかさになる。
- **流体外の除外:** 深度 0 の画素は重みに入れない。これで輪郭が内側に縮まない。
- **分離可能:** 2D のフィルタ（ R^2 タップ）を横 1D + 縦 1D（ $2R$ タップ）に分ける。厳密には等価でないが十分近く、はるかに速い。横→縦を 4 回反復する。

shaders/FluidSurface.hlsl

```
//-----
// 関数名 : PSBlur
// 概要   : 1 方向 (Params.zw で指定) のバイラテラルぼかし
//         アルゴリズム:
//         radiusPx = ワールド半径を画面に投影したピクセル数 (近いほど大きい)
//         重み w = exp(-k^2 / 2σs^2) * exp(-(d_k - d_0)^2 / 2σd^2)
//         出力   = Σ w d_k / Σ w
// 引数   : i : フルスクリーン三角形の補間値 (posH.xy がピクセル座標)
// 戻り値 : ぼかした深度
//-----
float PSBlur(FullscreenVSOut i) : SV_Target
{
    int2 pix = int2(i.posH.xy);
    float d0 = DepthTex.Load(int3(pix, 0));
    if (d0 <= 0.0)
        return 0.0; // 流体がないピクセル

    // ワールド半径 → ピクセル半径 (透視投影: 画面上の大きさ ∝ 1/深度)
    float radiusPx = BLUR_WORLD_RADIUS / d0 * Proj._m11 * ScreenSize.y * 0.5;
    radiusPx = clamp(radiusPx, 2.0, float(BLUR_MAX_TAPS));

    float sigmaS = radiusPx * 0.5; // 空間方向の標準偏差 [px]
    float invS2 = 1.0 / (2.0 * sigmaS * sigmaS);
```

```

float invD2 = 1.0 / (2.0 * BLUR_DEPTH_SIGMA * BLUR_DEPTH_SIGMA);

int2 dir = int2(Params.zw); // (1,0) または (0,1)
int2 maxPix = int2(ScreenSize.xy) - 1;

float sum = 0.0;
float wsum = 0.0;

[loop]
for (int k = -BLUR_MAX_TAPS; k <= BLUR_MAX_TAPS; ++k)
{
    if (abs(k) > radiusPx)
        continue;

    int2 q = clamp(pix + dir * k, int2(0, 0), maxPix);
    float d = DepthTex.Load(int3(q, 0));
    if (d <= 0.0)
        continue; // 流体外は無視（輪郭が内側へ縮まないように）

    float dd = d - d0;
    float w = exp(-float(k * k) * invS2) * exp(-dd * dd * invD2);
    sum += d * w;
    wsum += w;
}
return sum / max(wsum, 1e-6);
}

```

σ_d の決め方 最初は 0.35 m にしていましたが、球の凹凸（ ≈ 0.4 m）が平滑化されず「ブロック状」に見えました。流体外は別途除外しているので、 σ_d は球の凹凸より十分大きい 1.5 m にしてよく、それで滑らかな面になりました。「何と何を区別したいのか」を考えて閾値を決めるのがコツです。

15.4 法線の復元

ばかした深度から、各ピクセルのビュー空間位置を復元します。第 1 章の透視投影の式を逆向きに使います。

$$\mathbf{p} = (\text{ndc}_x \cdot d / P_{00}, \text{ndc}_y \cdot d / P_{11}, d)$$

隣接ピクセルとの位置の差（偏微分）の外積が法線です。輪郭では深度が飛ぶので、左右・上下それぞれで「差の小さい側」を採用して法線が壊れないようにします。ピクセルの +y は画面の下向き（ビュー空間の -y）なので、 $(+x) \times (-y) = -z$ がカメラ側を向く法線になります。

shaders/FluidSurface.hlsl

```

//-----
// 関数名 : ViewPosFromDepth
// 概要   : ピクセル座標と深度からビュー空間座標を復元する
//         NDC x = x_view * Proj._m00 / z_view → x_view = ndc.x * z / Proj._m00
// 引数   : pix : ピクセル座標（整数）
//         d    : そのピクセルのビュー空間深度
// 戻り値 : ビュー空間座標
//-----
float3 ViewPosFromDepth(int2 pix, float d)
{
    float2 ndc = (float2(pix) + 0.5) * ScreenSize.zw * 2.0 - 1.0;
    ndc.y = -ndc.y; // 画面座標は下向きが正、NDC は上向きが正
}

```

```

return float3(ndc.x * d / Proj._m00, ndc.y * d / Proj._m11, d);
}

```

15.5 水面の陰影と合成

復元した法線で、第 4 章の 3 つの成分を合成します。水らしさの決め手は**フレネル**で、斜めから見た部分ほど空の色が強く映ります。

拡散: $C_w(0.5 + 0.5 \max(0, \mathbf{n} \cdot \mathbf{L}))$, 鏡面: $0.35 \max(0, \mathbf{n} \cdot \mathbf{H})^{48}$, フレネル: $F = 0.06 + 0.94(1 - \mathbf{n} \cdot \mathbf{V})^4$

shaders/FluidSurface.hlsl

```

//-----
// 関数名 : PSComposite
// 概要   : 平滑化された深度から法線を求めて水面をシェーディングし、
//           アルファブレンドで地形の上に描く。
//           法線の求め方:
//           右隣・左隣との位置差 ddx, 上隣・下隣との位置差 ddy を計算し、
//           輪郭（深度が飛ぶ場所）を避けるため差の小さい方を採用して
//            $\mathbf{n} = \text{normalize}(\text{cross}(\mathbf{ddx}, \mathbf{ddy}))$  とする。
// 引数   : i : フルスクリーン三角形の補間値
// 戻り値 : 水面の色（アルファ付き）
//-----
float4 PSComposite(FullscreenVSOut i) : SV_Target
{
    int2 pix = int2(i.posH.xy);
    float d0 = DepthTex.Load(int3(pix, 0));
    if (d0 <= 0.0)
        discard;

    int2 maxPix = int2(ScreenSize.xy) - 1;
    float3 p = ViewPosFromDepth(pix, d0);

    // ---- x 方向の偏微分 ----
    float dR = DepthTex.Load(int3(min(pix.x + 1, maxPix), 0));
    float dL = DepthTex.Load(int3(max(pix.x - 1, 0), 0));
    float3 ddxR = ViewPosFromDepth(pix.x + 1, dR) - p;
    float3 ddxL = p - ViewPosFromDepth(pix.x - 1, dL);
    float3 ddx;
    if (dR <= 0.0) ddx = ddxL;
    else if (dL <= 0.0) ddx = ddxR;
    else ddx = (abs(ddxR.z) < abs(ddxL.z)) ? ddxR : ddxL;

    // ---- y 方向の偏微分 ----
    float dD = DepthTex.Load(int3(min(pix.y + 1, maxPix.y), 0));
    float dU = DepthTex.Load(int3(max(pix.y - 1, 0), 0));
    float3 ddyD = ViewPosFromDepth(pix.y + 1, dD) - p;
    float3 ddyU = p - ViewPosFromDepth(pix.y - 1, dU);
    float3 ddy;
    if (dD <= 0.0) ddy = ddyU;
    else if (dU <= 0.0) ddy = ddyD;
    else ddy = (abs(ddyD.z) < abs(ddyU.z)) ? ddyD : ddyU;

    // 画面の +x は右 (ビュー +x)、ピクセルの +y は下 (ビュー -y) なので
    // cross(ddx, ddy) = cross(+x, -y) = -z となり、カメラ側を向く法線になる
    float3 n = normalize(cross(ddx, ddy));
    if (n.z > 0.0) n = -n; // 念のためカメラ側を向かせる
}

```

```

// ---- シェーディング ----
float3 L = normalize(LightDirV.xyz);
float3 V = normalize(-p);
float3 H = normalize(L + V);

float diff = saturate(dot(n, L));
float spec = pow(saturate(dot(n, H)), 48.0);
float fresnel = 0.06 + 0.94 * pow(1.0 - saturate(dot(n, V)), 4.0); // Schlick の近似

const float3 waterColor = float3(0.64, 0.68, 0.78); // 参考動画の灰青色
const float3 skyColor = float3(0.90, 0.93, 0.98); // 反射する空の色

float3 col = waterColor * (0.50 + 0.50 * diff);
col = lerp(col, skyColor, fresnel * 0.5);
col += spec * 0.35;

float alpha = 0.90;
return float4(col, alpha);
}

```

$\alpha = 0.9$ のアルファブレンドで地形の上に重ねます。合成パスは深度テストなしで描きますが、深度パスの時点で地形との前後関係が解決済みなので、流体の深度がある画素は必ず見えている流体です。

src/FluidRenderer.cpp

```

//-----
// RenderSurfaceMode
// 1. 深度テクスチャ [0] をクリア (0 = 流体なし) し、メインの深度バッファと組で
//    球を描く → 地形に隠れる部分は自動的に除かれる
// 2. [0]→[1] 横ぼかし, [1]→[0] 縦ぼかし を m_blurIterations 回
// 3. バックバッファに [0] を入力として水面を合成 (アルファブレンド)
//-----
void FluidRenderer::RenderSurfaceMode(Graphics& gfx, UINT count, FrameConstants& frame, ID3D11Buffer*
cbFrame)
{
    ID3D11DeviceContext* ctx = gfx.GetContext();

    // ---- 1. 深度パス ----
    frame.Params.x = m_surfaceRadius;
    frame.Params.z = 0.0f;
    frame.Params.w = 0.0f;
    gfx.UpdateBuffer(cbFrame, &frame, sizeof(frame));

    const float clearZero[4] = { 0.0f, 0.0f, 0.0f, 0.0f };
    ctx->ClearRenderTargetView(m_depthRTV[0].Get(), clearZero);

    ID3D11RenderTargetView* rtv0 = m_depthRTV[0].Get();
    ctx->OMSetRenderTargets(1, &rtv0, gfx.GetDepthStencilView());
    DrawSpheres(gfx, count, m_psDepth.Get());

    // ---- 2. ぼかしパス (ピンポン) ----
    for (int it = 0; it < m_blurIterations; ++it)
    {
        // 横方向: [0] → [1]
        frame.Params.z = 1.0f; frame.Params.w = 0.0f;
        gfx.UpdateBuffer(cbFrame, &frame, sizeof(frame));
        ID3D11RenderTargetView* rtv1 = m_depthRTV[1].Get();
        ctx->OMSetRenderTargets(1, &rtv1, nullptr);
        DrawFullscreen(gfx, m_psBlur.Get(), m_depthSRV[0].Get());
    }
}

```

```

// 縦方向: [1] → [0]
frame.Params.z = 0.0f; frame.Params.w = 1.0f;
gfx.UpdateBuffer(cbFrame, &frame, sizeof(frame));
ctx->OMSetRenderTargets(1, &rtv0, nullptr);
DrawFullscreen(gfx, m_psBlur.Get(), m_depthSRV[1].Get());
}

// ---- 3. 合成パス ----
ID3D11RenderTargetView* back = gfx.GetBackBufferRTV();
ctx->OMSetRenderTargets(1, &back, nullptr);
ctx->OMSetBlendState(gfx.GetBlendAlpha(), nullptr, 0xFFFFFFFF);
DrawFullscreen(gfx, m_psComposite.Get(), m_depthSRV[0].Get());
ctx->OMSetBlendState(gfx.GetBlendOpaque(), nullptr, 0xFFFFFFFF);

// 後続の描画のために深度バッファ付きのバックバッファに戻す
ctx->OMSetRenderTargets(1, &back, gfx.GetDepthStencilView());
}

```

コラム: レンダーターゲットのピンポン テクスチャは「読みながら同じものを書く」ことができません。ぼかしのように同じ画像を繰り返し加工するときは、2 枚のテクスチャ A, B を用意し A→B, B→A と交互に使います。これを「ピンポン」と呼びます。SRV（読み取り）と RTV（書き込み）を同時に同じテクスチャにバインドすると、デバッグレイヤーが警告を出します。

演習 15

1. `SetBlurIterations(0)` にして、ぼかしなしの「球がボコボコ並んだ面」を見てください。1, 2, 4 と増やして変化を観察しましょう。
2. `BLUR_DEPTH_SIGMA` を 0.2 にして、ブロック状の見た目を再現してください。
3. 法線の「差の小さい側を採用」をやめて常に右側・下側を使うと、輪郭でどんなアーティファクトが出ますか。
4. 合成の α を 0.5 にして、透けた水を試してください。より本格的には「厚み」を別パスで蓄積して透過率に使います（第 18 章）。

カメラ・入力・メインループ

これまでの部品を 1 つのアプリケーションにまとめる「骨組み」を学びます。Windows のウィンドウとメッセージ、入力の受け取り、時間の管理、そして物理と描画を毎フレーム呼ぶループの構造です。

16.1 ウィンドウとメッセージループ

Windows のアプリケーションは、OS から送られる「メッセージ」（キー入力、マウス移動、サイズ変更...）をウィンドウプロシージャで処理します。ゲームやシミュレーションでは、メッセージが無いときにも描画を続けたいので、`PeekMessage` で「メッセージがあれば処理、無ければ 1 フレーム描画」という形にします。

src/main.cpp

```
while (msg.message != WM_QUIT)
{
    if (PeekMessageW(&msg, nullptr, 0, 0, PM_REMOVE))
    {
        TranslateMessage(&msg);
        DispatchMessageW(&msg);
        continue;
    }

    // 経過時間
    const auto now = Clock::now();
    const float dt = std::chrono::duration<float>(now - prevTime).count();
    prevTime = now;

    // 物理更新
    if (!g_app.paused)
        g_app.sph.Update(dt);

    // 描画
    const float clearColor[4] = { 1.0f, 1.0f, 1.0f, 1.0f }; // 参考動画と同じ白背景
    g_app.gfx.BeginFrame(clearColor);
    UpdateFrameConstants();
    g_app.terrain.Render(g_app.gfx);
    g_app.fluidRenderer.Render(g_app.gfx, g_app.sph.GetRenderData(), g_app.mode,
                              g_app.frame, g_app.cbFrame.Get());
    g_app.gfx.EndFrame(true);

    // FPS 計測 (0.5 秒ごとに更新)
    g_app.fpsTimer += dt;
    g_app.fpsFrames++;
    if (g_app.fpsTimer >= 0.5f)
    {
        g_app.fps = g_app.fpsFrames / g_app.fpsTimer;
        g_app.fpsTimer = 0.0f;
        g_app.fpsFrames = 0;
    }

    // タイトル更新 (0.25 秒ごと)
    titleTimer += dt;
```

```

    if (titleTimer >= 0.25f)
    {
        titleTimer = 0.0f;
        UpdateTitle(hWnd);
    }
}

```

- 経過時間 `dt` は `steady_clock`（単調増加する高分解能時計）で測ります。壁時計（`system_clock`）は時刻合わせで巻き戻ることがあるので使いません。
- 物理更新は `dt` に応じてサブステップ数が決まり（第 10 章）、描画は 1 回です。

16.2 入力処理

src/main.cpp

```

//-----
// 関数名 : WndProc
// 概要   : ウィンドウメッセージの処理（入力・リサイズ・終了）
// 引数   : Win32 標準
// 戻り値 : メッセージ処理結果
//-----
LRESULT CALLBACK WndProc(HWND hWnd, UINT msg, WPARAM wParam, LPARAM lParam)
{
    switch (msg)
    {
        case WM_DESTROY:
            PostQuitMessage(0);
            return 0;

        case WM_SIZE:
            if (g_app.initialized && wParam != SIZE_MINIMIZED)
            {
                const int w = LOWORD(lParam);
                const int h = HIWORD(lParam);
                if (g_app.gfx.Resize(w, h))
                    g_app.fluidRenderer.OnResize(g_app.gfx);
            }
            return 0;

        case WM_KEYDOWN:
            switch (wParam)
            {
                case VK_ESCAPE: PostQuitMessage(0); break;
                case '1':       g_app.mode = FluidRenderMode::Particles; break;
                case '2':       g_app.mode = FluidRenderMode::Surface; break;
                case VK_SPACE:  g_app.paused = !g_app.paused; break;
                case 'R':       g_app.sph.Reset(); break;
                default: break;
            }
            return 0;

        case WM_LBUTTONDOWN:
            g_app.lButtonDown = true;
            g_app.lastMouseX = GET_X_LPARAM(lParam);
            g_app.lastMouseY = GET_Y_LPARAM(lParam);
            SetCapture(hWnd);
            return 0;
    }
}

```

```

case WM_RBUTTONDOWN:
    g_app.rButtonDown = true;
    g_app.lastMouseX = GET_X_LPARAM(lParam);
    g_app.lastMouseY = GET_Y_LPARAM(lParam);
    SetCapture(hWnd);
    return 0;

case WM_LBUTTONDOWN:
case WM_RBUTTONUP:
    g_app.lButtonDown = false;
    g_app.rButtonDown = false;
    ReleaseCapture();
    return 0;

case WM_MOUSEMOVE:
{
    const int x = GET_X_LPARAM(lParam);
    const int y = GET_Y_LPARAM(lParam);
    const int dx = x - g_app.lastMouseX;
    const int dy = y - g_app.lastMouseY;
    if (g_app.lButtonDown)
    {
        // 1 ピクセルあたり 0.3 度回転
        g_app.camera.Rotate(-dx * 0.005f, dy * 0.005f);
    }
    else if (g_app.rButtonDown)
    {
        g_app.camera.Pan(-dx * 0.05f, dy * 0.05f);
    }
    g_app.lastMouseX = x;
    g_app.lastMouseY = y;
    return 0;
}

case WM_MOUSEWHEEL:
{
    const int delta = GET_WHEEL_DELTA_WPARAM(wParam);
    g_app.camera.Zoom(delta > 0 ? 0.9f : 1.1f);
    return 0;
}

default:
    break;
}
return DefWindowProc(hWnd, msg, wParam, lParam);
}

```

`SetCapture` は、ドラッグ中にマウスがウィンドウの外に出てもメッセージを受け取り続けるための呼び出しです。`WM_SIZE` では、初期化が終わる前に届くメッセージを `initialized` フラグで無視しています。

16.3 リソースの検索

実行ファイルの場所は、Visual Studio から実行するときと `bin\` から直接起動するときで異なります。シェーダーやアセットは「カレントディレクトリ → exe の場所 → その親 → さらに親」の順に探し、どこから起動しても見つかるようにしています。

```
//-----
// 関数名 : FindProjectDirectories
// 概要   : shaders フォルダと assets フォルダを探して Graphics に登録する。
//         実行ファイルの場所が bin\ や bin\Release\ でも動くように、
//         カレントディレクトリ → exe のフォルダ → その親 → さらに親 の順に探す。
//         (shaders\Common.hlsl の存在で判定し、assets は同じ階層とみなす)
// 引数   : なし
// 戻り値 : true = shaders が見つかった (assets はなくてもよい: ノイズ生成にフォールバック)
//-----
bool FindProjectDirectories()
{
    wchar_t exePath[MAX_PATH] = {};
    GetModuleFileNameW(nullptr, exePath, MAX_PATH);
    std::wstring exeDir(exePath);
    const size_t slash = exeDir.find_last_of(L"\\");
    if (slash != std::wstring::npos)
        exeDir = exeDir.substr(0, slash);

    const std::wstring roots[] = {
        L".",
        exeDir,
        exeDir + L"\\..",
        exeDir + L"\\..\\..",
    };
    for (const auto& root : roots)
    {
        if (FileExists(root + L"\\shaders\\Common.hlsl"))
        {
            Graphics::SetShaderDirectory(root + L"\\shaders");
            Graphics::SetAssetDirectory(root + L"\\assets");
            return true;
        }
    }
    return false;
}
```

演習 16

1. キー 3 で「粒子と表面を同時に描く」モードを追加してみましょう (`FluidRenderer::Render` を 2 回呼ぶ) 。
2. マウスの中ボタンドラッグでカメラの注視点を上下させる操作を追加してください。
3. タイトルバーの代わりに、フレーム時間の内訳 (物理 / 描画) を計測して表示してみましょう。

第 V 部

発展編

速くする、壊れない、その先へ。

性能とデバッグ

動くものを「速く」「壊れにくく」するための実践的な知識をまとめます。このプログラムの開発過程で実際に起きた問題と、その解決を例にします。

17.1 性能の推移

段階	粒子数	FPS	変更点
初期実装	8,300	20～24	密度・力の各パスで近傍を走査。フレーム時間に応じてサブステップが増え、遅くなるほどさらに遅くなる
近傍リスト + マスク + 上限	10,500	106～116	近傍走査を 1 回に、剰余をビットマスクに、サブステップ上限 5
安定化追加後（粒子）	12,000	63～120	流れが密になり近傍数が増加
同（表面生成）	12,000	54～80	ぼかし 8 パス分の GPU 負荷

17.2 ボトルネックの見つけ方

- CPU か GPU か: 粒子モードと表面モードで FPS が違えば、その差は GPU。物理を一時停止（Space）して FPS が跳ね上がれば CPU（物理）がボトルネック。
- 計測を入れる: `steady_clock` で各段（格子構築 / 密度 / 力 / 積分 / 描画）の時間を測り、タイトルに出す。推測より計測。
- スケーリングを見る: 粒子数を 2 倍にして時間が 2 倍なら線形、4 倍なら $O(N^2)$ の部分が残っている。

17.3 数値的な安定性

症状	原因と対処
粒子が爆発的に飛散する	Δt が CFL 条件を超えている / k が大きすぎる。 Δt を下げるか k を下げる。速度・加速度の上限を安全弁にする
粒子が塊になる	負圧（引力）が働いている。圧力を $\max(, 0)$ で打ち切る。poly6 の勾配を圧力に使っていないか確認
壁際で粒子が弾かれる	境界での密度急上昇。加速度クランプ、接触ゾーン減衰（第 12 章）
処理落ちすると加速度的に遅くなる	サブステップ数の上限を設け、実時間より遅く進める
NaN が伝播して全滅する	0 除算（ $r = 0$ の粒子対、密度 0）。 $r^2 < \varepsilon$ の対を除外、密度の下限

17.4 描画のデバッグ

- **何も映らない:** カリングを無効に → 映れば巻き順。それでも映らなければ行列（転置忘れ、near/far）やビューポート。
- **D3D デバッグレイヤー:** Debug ビルドで有効化。SRV と RTV の同時バインド、未設定のステート、入力レイアウトの不一致などを出力ウィンドウに警告する。
- **中間結果を見る:** 法線を色として出す、深度テクスチャをそのまま表示する、ぼかし回数を 0 にする。パスを 1 つずつ確かめる。
- **シェーダーのコンパイルエラー:** メッセージボックスに行番号つきで出る。HLSL は暗黙の型変換に厳しい（`int2` と `float2` の混在など）。

コラム: 「最初は動かない」が普通 このプログラムの初回実行では、min/max マクロとの衝突でコンパイルが通らず、次に粒子が谷の外に飛散し、表面生成はブロック状でした。1 つずつ原因を切り分け、パラメータの意味を考えて直すことで完成しています。CG と物理シミュレーションの開発は、この「観察 → 仮説 → 修正」の繰り返しです。

発展課題

本書のプログラムを出発点に、さらに本格的な表現・性能を目指すための方向を示します。それぞれ、何が足りないのか・どの論文や手法が答えになるのかを整理します。

課題	現状の制約	アプローチ
粒子数を 10 万以上に	CPU SPH は 1~2 万が上限	コンピュータシェーダーへ移植。格子構築は GPU 基数ソート、密度・力はスレッドグループで並列。粒子データを GPU に置いたままにできる
非圧縮性の向上	WCSPH は圧縮率が k に依存し、 k を上げると Δt が小さくなる	PCISPH (Solenthaler 2009)、IISPH、DFSPH (Bender 2015)。圧力を反復で解き、大きな Δt で非圧縮を保つ
境界の精度	境界密度欠損を減衰で吸収	境界粒子 (Akinci 2012)。地形表面に粒子をサンプリングして密度・圧力に寄与させる
表面張力・飛沫	表面張力なし	Akinci らの凝集力モデル、飛沫粒子の二次シミュレーション
透明な水	α 一定、屈折なし	厚みパス (加算ブレンドで粒子の厚さを蓄積) → 透過率と屈折、背景の歪み
視点に依存しない水面	スクリーンスペースは見える面のみ	Marching Cubes、異方性カーネル (Yu & Turk 2013) による陽なメッシュ生成
洞窟・オーバーハング地形	高さマップ	符号付き距離場 (SDF) を構築し、最近傍面への押し戻しで衝突
大きな水面・海	粒子法は大領域に不向き	格子法 (浅水方程式、FLIP/PIC) とのハイブリッド

18.1 取り組む順番の提案

1. **厚みパスと透過**: 表面生成に 1 パス足すだけで見た目が大きく向上する。GPU の加算ブレンドの練習にもなる。
2. **境界粒子**: SPH の理解が深まる。地形の高さマップからサンプリングすれば実装は比較的容易。
3. **GPU SPH**: コンピュータシェーダー、構造化バッファの読み書き、GPU 上のソートと、GPGPU の基礎を一通り学べる。
4. **PCISPH / DFSPH**: 数値解法の理解が必要。論文を読みながら実装する。

数学のまとめ

A.1 ベクトル

$$|\mathbf{a}| = \sqrt{\mathbf{a} \cdot \mathbf{a}}, \quad \hat{\mathbf{a}} = \mathbf{a}/|\mathbf{a}|, \quad \mathbf{a} \cdot \mathbf{b} = |\mathbf{a}||\mathbf{b}|\cos\theta$$

$$\mathbf{a} \times \mathbf{b} = (a_y b_z - a_z b_y, a_z b_x - a_x b_z, a_x b_y - a_y b_x), \quad |\mathbf{a} \times \mathbf{b}| = |\mathbf{a}||\mathbf{b}|\sin\theta$$

$$\mathbf{a} \text{ の } \mathbf{n} \text{ 方向成分: } (\mathbf{a} \cdot \mathbf{n})\mathbf{n}, \quad \text{反射: } \mathbf{a} - 2(\mathbf{a} \cdot \mathbf{n})\mathbf{n}$$

A.2 変換

クリップ座標 = $\mathbf{v} \cdot \text{World} \cdot \text{View} \cdot \text{Proj}$ (DirectXMath の行ベクトル規約)

NDC = clip.xyz / clip.w, 画面 $x = (\text{ndc}_x + 1)/2 \cdot \text{幅}$, 画面 $y = (1 - \text{ndc}_y)/2 \cdot \text{高さ}$

逆変換 (ビュー空間) : $x = \text{ndc}_x z / P_{00}$, $y = \text{ndc}_y z / P_{11}$

A.3 補間

$$\text{lerp}(a, b, t) = a + (b - a)t, \quad \text{smoothstep: } t^2(3 - 2t), \quad \text{fade: } t^3(t(6t - 15) + 10)$$

$$\text{双線形: } \text{lerp}(\text{lerp}(h_{00}, h_{10}, t_x), \text{lerp}(h_{01}, h_{11}, t_x), t_z)$$

$$\text{重心座標: } \mathbf{P} = w_0 \mathbf{A} + w_1 \mathbf{B} + w_2 \mathbf{C}, \quad w_0 + w_1 + w_2 = 1$$

A.4 SPH

$$\rho_i = \sum_j m W_{\text{poly6}}(r_{ij}), \quad p_i = k \max(\rho_i - \rho_0, 0)$$

$$\mathbf{a}_i = [\sum_j m(p_i + p_j)/(2\rho_j) \cdot 45/(\pi h^6)(h - r)^2 \hat{\mathbf{r}}_{ij} + \mu \sum_j m(\mathbf{v}_j - \mathbf{v}_i)/\rho_j \cdot 45/(\pi h^6)(h - r)] / \rho_i + \mathbf{g}$$

$$\text{CFL: } \Delta t < 0.4 h / (\sqrt{k} + v_{\max})$$

ファイル構成と API 要約

ファイル	内容	主な章
src/main.cpp	ウィンドウ、入力、メインループ、定数バッファ更新	3, 16
src/Graphics.h/.cpp	D3D11 初期化、ステート、シェーダーコンパイル、バッファ更新	2, 3
src/FrameConstants.h	共通定数バッファ構造体	3
src/Camera.h/.cpp	オービットカメラ	1
src/Noise.h/.cpp	パーリンノイズ / fBm / 尾根 / 周期版	5
src/Terrain.h/.cpp	高さマップ、メッシュ、テクスチャ、OBJ 取り込み、衝突用問い合わせ	6, 7, 8
src/TextureLoader.h/.cpp	WIC 画像読み込み、ミップ付きテクスチャ作成	7
src/MeshLoader.h/.cpp	OBJ 読み込み	8
src/SPH.h/.cpp	SPH 本体、近傍探索、境界、エミッタ	10~13
src/FluidRenderer.h/.cpp	粒子描画、表面生成	14, 15
shaders/Common.hlsl	定数バッファ、カラーマップ、全画面 VS	3, 14
shaders/Terrain.hlsl	地形の VS/PS	3, 4
shaders/Particle.hlsl	ビルボード VS、球の色 / 深度 PS	14, 15
shaders/FluidSurface.hlsl	ぼかし、法線復元、合成	15
assets/rock.png, terrain.obj	岩テクスチャ、地形メッシュ（差し替え可）	7, 8
tools/*.py	Blender 用 / 単体の試作地形生成スクリプト	8
docs/API.md, FLOW.md	API 仕様書、Mermaid フロー図（詳細はこちら）	—

主要クラスの公開関数

クラス	関数
Graphics	Initialize, Resize, BeginFrame, EndFrame, CompileShaderFromFile, CreateConstantBuffer, UpdateBuffer, Get*State, ShaderPath, AssetPath
Camera	Rotate, Zoom, Pan, GetPosition, GetViewMatrix, GetProjMatrix
Terrain	Initialize, Render, GetHeight, GetNormal, GetValleyCenterX, IsMeshFromFile, IsTextureFromFile
SPH	Initialize, Reset, Update, GetRenderData, GetParticleCount, GetSimTime
FluidRenderer	Initialize, OnResize, Render, SetParticleRadius, SetSurfaceRadius, SetBlurIterations
Noise / TextureLoader / MeshLoader	Perlin2D, Fbm2D, Ridged2D, *Tileable / LoadFromFile, CreateFromPixels / LoadObj, ComputeSmoothNormals, ComputeBounds

用語集

用語	説明
NDC（正規化デバイス座標）	透視除算後の座標。 x, y が $-1 \sim 1$ 、深度が $0 \sim 1$ の範囲に正規化されている
ラスタライズ	三角形をピクセルの集まりに変換すること。頂点属性は重心座標で補間される
深度バッファ / 深度テスト	ピクセルごとに最も手前の深度を記録し、奥のものを描かないようにする仕組み
巻き順（ワインディング）	三角形の頂点の並び順。画面上で時計回りか反時計回りかで表裏を判定する
RTV / DSV / SRV	レンダーターゲット / 深度ステンシル / シェーダーリソースの各ビュー。リソースの用途を示す
セマンティクス	HLSL の入出力に付ける意味のラベル（ <code>SV_POSITION</code> など）
定数バッファ	フレーム中に一定の値をシェーダーへ渡すバッファ。16 バイト境界
StructuredBuffer	構造体の配列としてシェーダーから読めるバッファ
インスタンスング	1 回の描画命令で同じ形状を多数描く仕組み
ビルボード	常にカメラに正対する四角形
インポスター	単純な形状をシェーダーで複雑な形に見せかける技法
ミップマップ	テクスチャの縮小版の系列。遠景のちらつきを防ぐ
fBm	周波数と振幅を変えたノイズの重ね合わせ
高さマップ	(x, z) ごとに高さを持つ 2 次元配列による地形表現
SPH	Smoothed Particle Hydrodynamics。粒子とカーネルで流体を離散化する手法
カーネル関数	距離に応じた重み。半径 h の外で 0
状態方程式	密度から圧力を直接決める式（WCSPH）
CFL 条件	時間刻みの上限。情報が 1 ステップでセルより遠くへ伝わってはならない
シンプレクティック・オイラー法	速度を先に更新し、その速度で位置を更新する積分法
空間ハッシュ	セル座標を固定サイズの表に写す近傍探索の手法
カウンティングソート	キーの出現数と累積和で並べ替える、 $O(N)$ のソート
バイラテラルフィルタ	距離と値の両方の近さで重み付けする、エッジを保つ平滑化
スクリーンスペース	3D ではなく画面のピクセル上で処理を行うこと
フレネル反射	視線と面の角度で反射率が変わる現象

参考文献

1. M. Müller, D. Charypar, M. Gross, "Particle-Based Fluid Simulation for Interactive Applications," SCA 2003. — SPH を CG に導入した基本文献。カーネル・状態方程式は本書と同じ。
2. W. J. van der Laan, S. Green, M. Sainz, "Screen Space Fluid Rendering with Curvature Flow," I3D 2009. — スクリーンスペース流体レンダリングの原典。
3. K. Perlin, "Improving Noise," SIGGRAPH 2002. — Fade 曲線と勾配の改良。
4. M. Teschner et al., "Optimized Spatial Hashing for Collision Detection of Deformable Objects," VMV 2003. — 空間ハッシュ関数。
5. N. Akinci et al., "Versatile Rigid-Fluid Coupling for Incompressible SPH," SIGGRAPH 2012. — 境界粒子。
6. B. Solenthaler, R. Pajarola, "Predictive-Corrective Incompressible SPH," SIGGRAPH 2009.
7. J. Bender, D. Koschier, "Divergence-Free SPH for Incompressible and Viscous Fluids," TVCG 2017.
8. J. Yu, G. Turk, "Reconstructing Surfaces of Particle-Based Fluids Using Anisotropic Kernels," TOG 2013.
9. C. Schlick, "An Inexpensive BRDF Model for Physically-based Rendering," Eurographics 1994.
10. D. Koschier, J. Bender, B. Solenthaler, M. Teschner, "Smoothed Particle Hydrodynamics Techniques for the Physics Based Simulation of Fluids and Solids," Eurographics Tutorial 2019. — SPH の現代的な総説。
11. Microsoft, "Direct3D 11 Graphics" / "HLSL" ドキュメント (learn.microsoft.com) 。